

Gymnázium Františka Křížíka a základní škola s.r.o.

oktáva

Šimon Brandner

Využití protokolu Matrix pro videohovory

Ročníková práce

Vedoucí práce: Ing. Jan Ježek

Plzeň 2024

Poděkování

Rád bych mnohokrát poděkoval panu Ing. Janu Ježkovi za vedení mé práce a panu Mgr. Petru Ustohalovi za její zaštitění. Také bych chtěl poděkovat mým kolegům ze společnosti Element, kteří mě toho mnoho naučili, a mé rodině a přátelům za dlouhodobou podporu.

Anotace

V posledních letech se značně rozšířilo využívání aplikací určených pro videokonference po internetu. K jejich rozvoji výrazně přispěla pandemie covid-19. Trend v jejich využívání však zůstává zachován a stávají se běžným nástrojem pro komunikaci. Většina aplikací, která poskytuje tyto funkce, je ale buď proprietární (closed-source), např. Zoom, Google Meet, MS Teams, či protokol, který používají, není otevřený, např. Jitsi. V této práci se zaměříme na otevřený protokol Matrix pro komunikaci v reálném čase, jehož součástí bude skupinové volání (dále MatrixRTC). V současné době je tato podpora pro skupinové volání v pokročilé fázi vývoje. Cílem této práce je rozebrat jednotlivé komponenty aplikací pro skupinové volání a prozkoumat samotný protokol MatrixRTC a jeho implementace, zhodnotit jeho použitelnost, flexibilitu, rozšiřitelnost atp. V praktické části je poté cílem navrhnout a implementovat tzv. break-out rooms pomocí MatrixRTC.

Klíčová slova

Matrix, WebRTC, VoIP, otevřený software

Annotation

In recent years, the use of applications designed for video conferencing over the Internet has become more common. The COVID-19 pandemic has contributed significantly to their development. Even with the pandemic over, the trend in their use remains and they are becoming a common tool for communication. Most applications that provide these features are either proprietary (closed-source), e.g. Zoom, Google Meet, MS Teams, or the protocol they use is not open, e.g. Jitsi. In this paper, we focus on the open protocol Matrix for real-time communication, which will include a group calling feature (hereafter MatrixRTC). Currently, the support for group calls is at an advanced stage of development. The aim of this paper is to analyse the different components of group calling applications and examine the MatrixRTC protocol itself and its implementations, to evaluate its usability, flexibility, extensibility etc. In the practical part, the goal is then to design and implement break-out rooms using MatrixRTC.

Keywords

Matrix, WebRTC, VoIP, open source

Obsah

1	Úvod	1
2	Protokol WebRTC	3
2.1	Spojování	3
2.1.1	NAT	4
2.1.2	STUN	5
2.1.3	TURN	6
2.1.4	ICE	7
2.1.5	Způsobí přechod na IPv6 zánik ICE a STUN?	8
2.2	Přenos audio a videa	8
2.2.1	RTP	8
2.2.2	RTCP	8
2.3	Posílání dat	8
2.4	Šifrování	9
2.4.1	DTLS	9
2.4.2	SRTP	9
2.5	Signalling	9
3	Skupinové volání pomocí WebRTC	14
3.1	Full-mesh	14
3.2	Servery – foci	15
3.2.1	Multipoint Control Unit	15
3.2.2	Selective Forwarding Unit	16
4	Další VoIP technologie	18
4.1	Protokol SIP	18
4.2	LiveKit	18
4.3	Protokol aplikace Zoom	18
4.4	Protokol aplikace Discord	18
4.5	Protokol aplikace Jitsi	19
5	Matrix	20
5.1	Specifikace a její změny	20
5.2	Primitiva	21
5.2.1	Uživatelé	21
5.2.2	Zařízení	21
5.2.3	Místnosti a události	22

5.2.4	To-device messages	22
5.3	Klienti	22
5.4	Homeservery	23
5.5	Přemostění (bridging)	23
5.6	End-to-end šifrování	23
6	MatrixRTC	25
6.1	Využití Matrix primitiv	25
6.1.1	Hovory mezi dvěma uživateli	25
6.1.2	Skupinové hovory	29
6.2	Klientské SDK	30
6.2.1	Matrix JavaScript SDK	30
6.2.2	Hydrogen SDK	31
6.3	Foci	31
6.3.1	Waterfall	33
7	Implementace break-out rooms	34
7.1	Návrh na doplnění specifikace	34
7.2	Samotná implementace	35
7.2.1	Matrix JavaScript SDK	35
7.2.2	Element Call	38
8	Budoucnost	40
8.1	Využití MatrixRTC v non-VoIP aplikacích	40
8.1.1	Third Room – virtuální světy postavené na Matrixu	40
8.1.2	TheBoard – virtuální tabule	40
8.2	Web Transport – alternativa k WebRTC	40
8.3	Digital Markets Act	40
9	Závěr	42
10	Seznam zkratk	43
11	Seznam obrázků	46
12	Reference	47

1 Úvod

29. října 1969 se zrodil **Advanced Research Projects Agency Network (ARPANET)**, předchůdce dnešního internetu. Byly poslány znaky „L“ a „O“. Student UCLA Charley Kline se totiž snažil napsat zprávu „LOGIN“, ale systém se po zadání dvou znaků zhroutil. Přibližně po hodině se mu konečně podařilo poslat celé slovo (McDowall, 2015). Trvalo to jen dva roky, než Ray Tomlinson poslal první e-mail, který zněl „something like QWERTYUIOP“ (Shontell, 2013).

V roce 1974 byl proveden první tzv. **Voice over Internet Protocol (VoIP)** hovor po síti **ARPANET** (DigiFone, [b.r.]). V roce 1989 v CERNu britský vědec Tim Berners-Lee stvořil World Wide Web (WWW) (CERN, [b.r.]). O dva roky později vznikla první **VoIP** aplikace *Speak Freely* (DigiFone, [b.r.]) a po dalších dvou letech se objevila první webkamera, která snímala konvici na kávu v počítačové laboratoři na University of Cambridge (Kesby, [b.r.]).

29. srpna 2003 byla vydána první verze aplikace *Skype* (Tänavsuu, 2018) a sloveso „skypovat“ se během dalších let běžnou součástí slovníku mnoha lidí. Často je používáno prakticky jako synonymum pro **VoIP** hovory a jeho běžnost užívání ukazuje na to, že **VoIP** aplikace se staly běžnou součástí našich životů.

Využití **VoIP** aplikací zaznamenalo další „boom“ v době pandemie covidu-19, kdy bylo mnoho lidí nuceno pracovat z domova, a tak se tyto nástroje staly nepostradatelnými (DeBari, 2021).

Dnes většina **VoIP** aplikací, jako je např. Microsoft Teams, Discord, Google Meet, Jitsi Meet atp., využívá protokolu **Web Real-Time Communication (WebRTC)**, který se stal standardem pro komunikaci na internetu v reálném čase (Ullal, 2021; Vass, 2018; 8x8, [b.r.(b)]; Google, [b.r.(b)]). Těmito aplikacemi se budeme zabývat v části 2. Výše uvedený software oproti prvním **VoIP** aplikacím (jako byla *Speak Freely*) přinesl mnoho zlepšení, mimo jiné např. umožňuje skupinové hovory až tisíců uživatelů (Microsoft, [b.r.(a)]). To, jak je toho docíleno, shrneme v části 3. A samotným aplikacím se pak budeme věnovat v části 4.

Přes mnoho zlepšení, ke kterým bezpochyby došlo, však v jistých ohledech dochází spíše ke zhoršení. Například protokoly, které používáme pro e-mailovou komunikaci, umožňují tzv. *federaci*. V tomto kontextu federace znamená, že jednotlivé e-mail servery spolu mohou komunikovat, a je tedy možné poslat e-mail od jednoho e-mailového poskytovatele k druhému (např. uživatelé Gmailu a Seznamu si spolu mohou bez jakýchkoliv potíží vyměňovat e-maily) (The Matrix.org Foundation, [b.r.(b)]). Podobně na tom byly kdysi i aplikace Yahoo! Messenger a MSN Messenger (Mook, 2005). O většině dnešních aplikací určených pro chat však to samé říct nemůžeme. Dnes už asi nikoho nenapadne, že by si uživatel Messengeru mohl vyměňovat zprávy s uživateli What's App (Lovejoy, 2022).

Existují ale protokoly jako *ActivityPub* (Lemmer-Webber; Tallon; Shepherd; Guy; Pro-

dromou, 2018), *XMPP* (XMPP Standards Foundation, [b.r.(a)]) a *Matrix* (The Matrix.org Foundation, [b.r.(c)]) (více v části 5), jejichž účelem je situaci změnit. Právě protokol Matrix bude dalším tématem, kterým se budeme zabývat, jelikož kromě běžných funkcí pro chat také umožňuje VoIP hovory pomocí WebRTC (více v části 6) (The Matrix.org Foundation, [b.r.(c)]; The Matrix.org Foundation, [b.r.(e)]).

Přestože protokol Matrix podporuje VoIP hovory, je tato část protokolu stále ve vývoji (Hodgson; Long; Townsend; Brandner, [b.r.]; Hodgson; Brandner; Abramov; Baker, [b.r.]) a chybí jí některé funkce, jako jsou např. break-out rooms. V části 7 se pokusíme navrhnout, jak do protokolu tuto funkci přidat a následně ji i implementujeme.

Nakonec, v části 8, se podíváme na to, jak by se situace mohla vyvíjet v dalších letech, a pokusíme se ukázat, že protokoly využívané pro komunikaci v reálném čase mají mnoho dalších využití kromě VoIP hovorů.

Aktuální verze práce (s jejím zdrojovým kódem) je veřejně dostupná na GitHubu. Lze ji nalézt na adrese github.com/SimonBrandner/RocnikovaPrace-MatrixRTC.

2 Protokol WebRTC

V této části práce vychází převážně z publikace [WebRTC For The Curious: Go beyond the APIs](#) (Bound; DuBois; voluntas, [\[b.r.\]](#)).

WebRTC je protokol, který umožňuje vytvoření bezpečného obousměrného spojení mezi dvěma klienty pro přeposílání audia, videa a libovolných dat v reálném čase. Toto spojení může být **peer-to-peer (P2P)**, ale data mohou být přeposílány i přes server, jak je popsáno v části 2.1. Je využíváno koncové šifrování, anglicky **end-to-end encryption (E2EE)**, což zaručuje, že server nemůže číst data, pokud jsou přes něj přeposílány (Bound; DuBois; voluntas, [\[b.r.\]](#)).

WebRTC spojení je složeno ze *stop (tracks)* a *datových kanálů (data channels)*. Stopa může být buď audio, či video (MDN contributors, 2023c). Tyto stopy mohou být přiřazeny do tzv. *streamů* (stream je často složen ze dvou stop – např. audia z mikrofону a videa z kamery) (MDN contributors, 2023b). Datové kanály se pak používají k přenosu libovolných dat (Google, 2020).

Application Programming Interface (API) protokolu **WebRTC** je specifikováno jen v JavaScriptu. Existují však i implementace **WebRTC** (např. **Pion** či **WebRTC.rs**) v jiných jazycích (Bound; DuBois; voluntas, [\[b.r.\]](#); Pion, [\[b.r.\]](#); WebRTC.rs, [\[b.r.\]](#)).

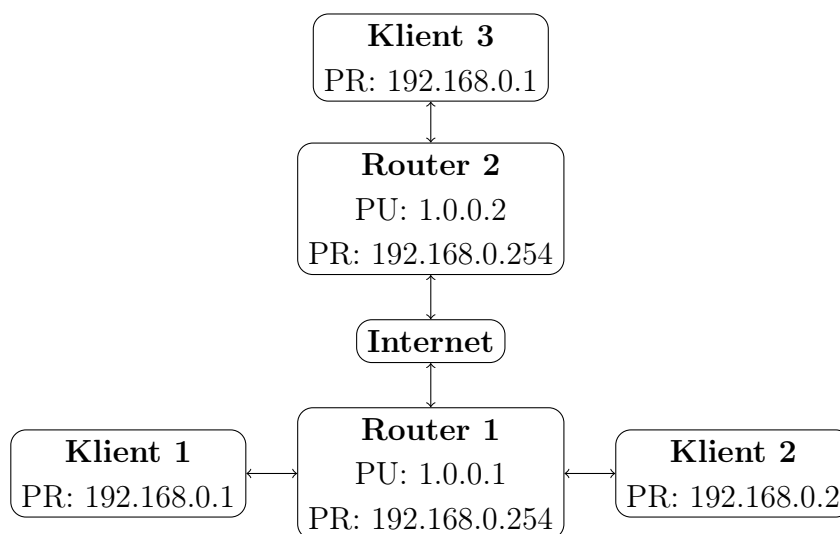
Protokol **WebRTC** se skládá z mnoha komponentů a využívá ke svému fungování mnoha jiných protokolů (Bound; DuBois; voluntas, [\[b.r.\]](#)). V této části si jednotlivé komponenty rozebereme a popíšeme, k čemu slouží.

2.1 Spojování

Tradiční aplikace často používají model klient-server, ve kterém má server fixovanou známou IP adresu, již může klient využít pro komunikaci. Ve **WebRTC** tomu ale tak není, neboť protokol je **P2P**. Oba klienti jsou tedy zodpovědní za vytvoření obousměrného spojení, což může být složité. Má to ale své výhody jako menší zpoždění. Vývojář se zároveň nemusí starat o žádné servery (Bound; DuBois; voluntas, [\[b.r.\]](#)).

Pro vytvoření spojení se používá protokolu **Interactive Connectivity Establishment (ICE)**. Protokol **ICE** slouží k nalezení nejlepšího způsobu, jak vytvořit spojení mezi dvěma klienty. Každý klient zveřejní své tzv. **ICE kandidáty**, což jsou v podstatě IP adresy a porty, které je možno využít pro komunikaci s ním (více v části 2.1.4) (Bound; DuBois; voluntas, [\[b.r.\]](#)). V této části se zabýváme jen samotným výběrem těchto kandidátů; jak si klienti předávají informace o kandidátech je rozebráno v sekci 2.5.

2.1.1 NAT



Obrázek 1: NAT (PU – veřejná IP adresa, PR – privátní IP adresa)

Kvůli nedostatku IPv4 adres je nutno reprezentovat několik zařízení v podsíti routeru jednou veřejnou IP adresou (Yu, 2018). Využívá se k tomu metody **Network Address Traversal (NAT)**, která vytváří mapování mezi veřejnou IP adresou¹ a portem a privátní IP adresou a portem v podsíti routeru (Bound; DuBois; voluntas, [b.r.]).

Na obrázku 1 vidíme situaci, ve které je snadné pro klienta 1 poslat paket klientovi 2, neboť jsou ve stejné podsíti. Ale pokud by klient 3 chtěl poslat paket klientovi za routerem 2, je nutné využít veřejných IP adres routerů. Ty ale klienti nemusí znát. Zároveň je nutné, aby router věděl, na jaký z klientů paket přeposlat (Bound; DuBois; voluntas, [b.r.]).

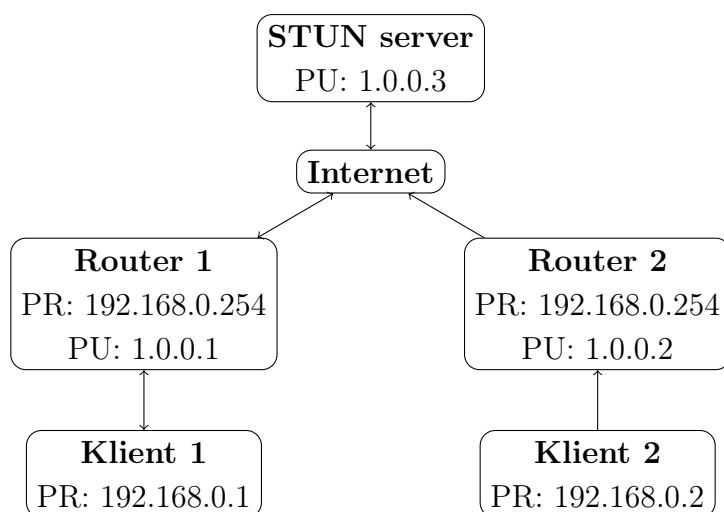
Jednou z možností mapování je *port forwarding*, jde o statické namapování předem definovaného portu na privátní IP adresu a port v podsíti routeru. Router 2 by mohl mít vytvořené takové mapování, že přijde-li paket na adresu 1.0.0.2:80 (tedy na port 80 na routeru 2), router 2 ho přesměruje na adresu 192.168.0.1:90 (port 90 na klientovi 3). Pošle-li klient 3 paket mimo podsít routeru 2, bude přeposlán routerem přes port 80 (příjemce bude tedy vidět 1.0.0.2:80 jako zdrojovou adresu). Toto mapování je vhodné pro případy, kdy chceme, aby dané zařízení za routerem bylo dostupné na IP adrese routeru na pevně daném portu (Busch, 2021).

Mapování ale nemusí být pevné a může existovat jen dočasně. Pošle-li klient 1 paket na 1.0.0.2:80 (port 80 na routeru 2), vybere router 1 nějaký port P, vytvoří mapování a přepošle paket z tohoto portu (příjemce bude tedy vidět 1.0.0.1:P jako zdrojovou

¹Pro zjednodušení předpokládáme statické veřejné IP adresy.

adresu). Když následně klient 3 pošle paket na 1.0.0.1:P (port P na routeru 1), router 1 rozpozná dle portu, že má paket zpět přeposlat na 192.168.0.1:80 (klienta 1, který posláním svého paketu mapování vytvořil). Toto mapování může ale v budoucnosti zaniknout². Tento typ mapování je nejčastější pro běžná zařízení (Bound; DuBois; voluntas, [b.r.]).

2.1.2 STUN



Obrázek 2: STUN (PU – veřejná IP adresa, PR – privátní IP adresa)

NAT sice umožňuje vytvořit mapování mezi veřejnou IP adresou a portem a IP adresou a portem v podsíti, ale klienti v podsíti neví, jaká je veřejná IP adresa routeru (obzvlášť pokud je dynamická), a pokud není použit port forwarding, neví ani, jaký port bude využit pro přeposílání paketů. **Session Traversal Utilities for NAT (STUN)** je protokol, který nám umožňuje tato omezení překonat (Bound; DuBois; voluntas, [b.r.]).

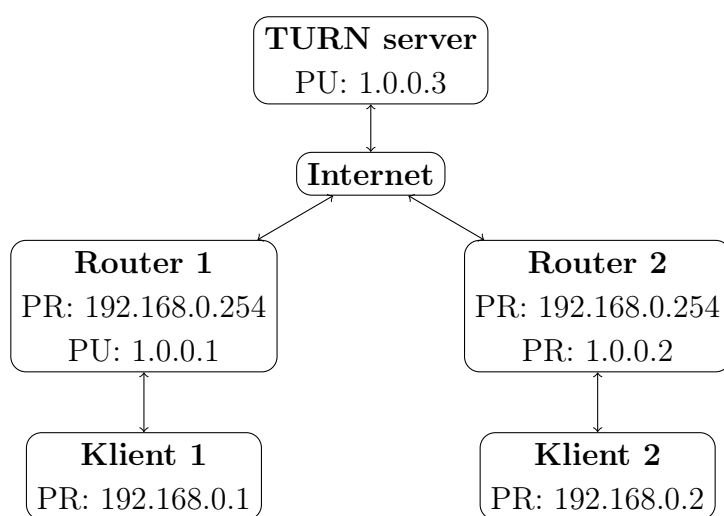
V síti, která je vidět na obrázku 2, klient 1 pošle **STUN Binding Request** na 1.0.0.3 (na **STUN** server, který má známou statickou veřejnou IP adresu). Router 1 využije metody **NAT** pro vytvoření mapování na určitém portu P. **STUN** server vidí, že paket přišel z adresy 1.0.0.1:P, tuto informaci pomocí **STUN Binding Response** pošle zpět na 1.0.0.1:P (router 1). Protože bylo na routeru 1 vytvořeno mapování, router 1 ví, že pokud přijde paket na port P, má jej přeposlat na IP adresu 192.168.0.1 (klientovi 1), ten poté ze **STUN Binding Response** může vyčíst, jaká je jeho veřejná IP adresa (někdy

²Jak dlouho je mapování funkční závisí na mnoha faktorech. Např. pokud je použit protokol **Transmission Control Protocol (TCP)**, mapování se ruší explicitně.

také *Mapped Address* či *Server Reflexive Candidate*). V podstatě se jedná o automatické dynamické port forwardování (Bound; DuBois; voluntas, [b.r.]).

Bohužel Mapped Address vůbec nemusí být užitečná, pokud je použito NAT mapování typu *Address Dependent*, což znamená, že router 1 přijímá na daném mapování pakety pouze z IP adresy 1.0.0.3 (ze STUN serveru)³. STUN naštěstí umožňuje zjištění typu NAT mapování, takže klient předem ví, jestli ho bude možné kontaktovat (Bound; DuBois; voluntas, [b.r.]).

2.1.3 TURN



Obrázek 3: TURN (PU – veřejná IP adresa, PR – privátní IP adresa)

Traversal Using Relays around NAT (TURN) je metoda, která umožňuje vytvořit spojení i v případě, že je NAT mapování Address Dependent, či pokud je klient za jiným druhem restriktivního firewallu. TURN lze také využít, přeje-li si klient, aby druhý klient neznal jeho IP adresu (Bound; DuBois; voluntas, [b.r.]).

TURN server funguje jako proxy (prostředník), přes které jsou preposílána data. Přeje-li si klient 1 na obrázku 3 využít TURN server, musí nejdříve vytvořit tzv. *alokaci*, která je následně využita k preposílání dat. Aby alokaci klient 1 vytvořil, musí serveru poskytnout uživatelské jméno, heslo, protokol (může být **User Datagram Protocol (UDP)** nebo **TCP**) a port (Bound; DuBois; voluntas, [b.r.]).

Podaří-li se alokaci vytvořit, odpověď TURN serveru obsahuje podobně jako u STUN Mapped Address (v našem případě 1.0.0.1:P1), *Lifetime* (což je doba, po kterou bude

³Address Dependent mapování lze často také nalézt pod pojmem *Symmetric NAT*. Tento pojem je ale nepřesný, neboť reálné NAT často nelze snadno klasifikovat do kategorií (Jennings; Audet, 2007).

alokace aktivní), *Relayed Address* (v našem případě 1.0.0.3:P2, což je adresa, kterou klient 1 předá ostatním klientům). Pokud jsou na Relayed Adresu (1.0.0.3:P2) posílány pakety, budou **TURN** serverem přeměřovány na Mapped Adresu (adresu, ze které byla vytvořena alokace) – 1.0.0.1:P1 (tedy router 1). Ten následně přepošle pakety na klienta 1 na základě mapování, které bylo vytvořeno při vytváření alokace (Bound; DuBois; voluntas, [b.r.]).

Nyní musí klient 2 poslat **TURN** serveru **STUN** Binding Request a klientovi 1 sdělit svou veřejnou adresu. Klient 1 pak může klientovi 2 povolit, aby na danou alokaci posílal pakety (Bound; DuBois; voluntas, [b.r.]).

Klient 1 může posílat na **TURN** server *Send Indication*, která obsahuje data, které se mají přeposlat klientovi 2 (Bound; DuBois; voluntas, [b.r.]).

Posílá-li se velké množství paketů, může být nevýhodné, aby každý z nich obsahoval IP adresu a port. Je tedy možné vytvořit tzv. *kanál* (*Channel*), mající své *ID kanálu* (*Channel ID*). Klient pak kanál používá v paketech místo IP adresy a portu. Paket je poté na **TURN** serveru upraven a IP adresa a port dodány (Bound; DuBois; voluntas, [b.r.]).

Vzhledem k tomu, že alokace jsou časově omezeny, klient může poslat *Refresh* request, který by měl prodloužit lifetime dané alokace (Bound; DuBois; voluntas, [b.r.]).

2.1.4 ICE

Interactive Connectivity Establishment (ICE) je protokol, který najde všechny páry adres a portů, které je možné využít pro spojení dvou klientů. Tyto páry se nazývají *páry kandidátů*. Může se jednat o privátní adresy v rámci podsítě routeru, Mapped Adresy (získané ze **STUN** serveru), či Relayed Adresy (získané z **TURN** serveru). Klienti komunikují pomocí **ICE** *ping* paketů, které umožňují ověřit funkčnost spojení (Bound; DuBois; voluntas, [b.r.]).

ICE klient je buď *controlling*, nebo *controlled*. Controlling klient je ten, který vybere vhodný pár kandidátů. Obvykle je controlling klient ten, který zahájil spojování (Bound; DuBois; voluntas, [b.r.]).

Nabízí-li každá strana 2 kandidáty, existují pak 4 páry kandidátů. Klienti využijí **ICE** ping pakety, aby ověřili, které páry jsou funkční. Ty páry, u kterých došlo k výměně dat, jsou pak označeny za *validní kandidáty*. Controlling klient následně vybere jeden z validních párů a označí ho za *nominovaný pár*. Klienti se pak znovu pokusí o obousměrnou komunikaci pomocí tohoto páru, a je-li úspěšná, stává se z daného párů *vybraný pár kandidátů* (Bound; DuBois; voluntas, [b.r.]).

Dojde-li po vytvoření spojení k potížím (např. vyprší **NAT** mapování nebo selže **TURN** server), celý proces začne znovu (Bound; DuBois; voluntas, [b.r.]).

2.1.5 Způsobí přechod na IPv6 zánik ICE a STUN?

NAT původně vzniklo jako způsob řešení nedostatku IPv4 adres, ale za léta svého vývoje se jejich význam rozšířil, neboť zároveň poskytují jednu z vrstev zabezpečení (Levent-Levi; Light, [b.r.]).

Přestože postupný přechod na IPv6 je nevyhnutelný, je nepravděpodobné, že by technologie jako **NAT** zmizely. Je tak smysluplné předpokládat, že technologie jako **ICE** a **STUN** rovněž zůstanou relevantní (Levent-Levi; Light, [b.r.]).

2.2 Přenos audio a videa

WebRTC umožňuje posílat neomezené množství audio a video stop, které buď mohou být nezávislé, nebo mohou být součástí streamu. Typickým příkladem je uživatel, který v jednom streamu posílá audio stopu ze svého mikrofону a video stopu ze své kamery a v druhém streamu audio stopu a video stopu ze svého počítače. Dalším příkladem je server, který přeposílá streamy jiných uživatelů, jak je popsáno v části 3 (Bound; DuBois; voluntas, [b.r.]).

2.2.1 RTP

Real-time Transport Protocol (RTP) je protokol, který se využívá pro přenos audia a videa. **RTP** umožňuje posílání několika streamů (ve **WebRTC** je **RTP** stream jedna stopa) přes jedno peer-spojení. **RTP** zároveň dává vývojáři informace o časování a pořadí paketů, je ale nezávislé na kodeku (Bound; DuBois; voluntas, [b.r.]).

2.2.2 RTCP

RTP Control Protocol (RTCP) je protokol, který umožňuje přeposílání metadat o **RTP** spojení. Obvykle se jedná o metadata, jako je např. ztráta paketů (Bound; DuBois; voluntas, [b.r.]).

2.3 Posílání dat

Stream Control Transmission Protocol (SCTP) je protokol, který umožňuje přeposílání libovolných dat pomocí **WebRTC**. **WebRTC datové kanály** (*data channels*) jsou jeho abstrakcí. Každé **WebRTC** spojení může obsahovat neomezeně datových kanálů (podobně jako je to u **RTP** streamů) (Bound; DuBois; voluntas, [b.r.]).

2.4 Šifrování

Všechna data přeposílána v rámci **WebRTC** spojení jsou **E2EE** (Bound; DuBois; voluntas, [b.r.]), což je zajištěno pomocí dvou protokolů, které si popíšeme níže.

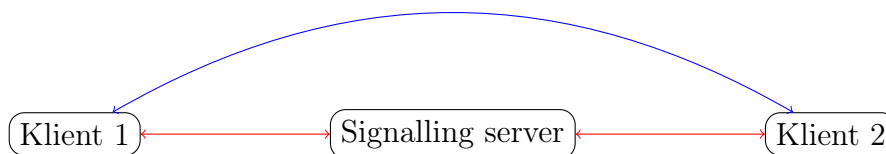
2.4.1 DTLS

Datagram Transport Layer Security (DTLS) je protokol, jenž generuje klíče pro šifrování dat, která jsou posílána přes protokol **UDP**. Je podobný protokolu **Transport Layer Security (TLS)**, který se využívá k šifrování dat přeposílaných **TCP** (Bound; DuBois; voluntas, [b.r.]).

2.4.2 SRTP

Secure Real-time Transport Protocol (SRTP) je protokol, který využívá externích klíčů vygenerovaných **DTLS** k šifrování **RTP** paketů (Bound; DuBois; voluntas, [b.r.]).

2.5 Signalling



Obrázek 4: Signalling

Signalling je zásadní součást **WebRTC**. Jedná se o proces při kterém si klienti vyměňují pomocí tzv. *signalling serveru* **ICE** kandidáty a informace o tom, jaká data plánují posílat, jaké kodeky⁴ podporují atp. Zprávy, které si klienti pomocí signalling serveru vyměňují, se nazývají *session descriptions*. Jsou ve formátu, který je definován protokolem zvaným **Session Description Protocol (SDP)** (místo *session description* se někdy píše jen **SDP**) (Bound; DuBois; voluntas, [b.r.]). Signalling server není nijak standardizován, jedinou podmínkou je, že realizuje výměnu **SDP**. To vede k tomu, že aplikace často nejsou mezi sebou kompatibilní (MDN contributors, [b.r.(b)]).

Chce-li klient 1 volat klienta 2, vygeneruje tzv. *session description offer* (**SDP offer**), kterou pomocí signalling serveru přepoše klientovi 2. Chce-li klient 2 hovor přijmout,

⁴Kodek je hardware či software, který se využívá pro zakódování, či rozkódování médií (Halavais, 2022; Gillis, 2022).

vygeneruje tzv. *session description answer* (*SDP answer*), kterou naopak přepośle klientovi 1. Poté, co dojde k výměně session descriptions, se protokol ICE pokusí vybrat vhodný pár kandidátů. Tento proces se nazývá *negotiation* (Bound; DuBois; voluntas, [b.r.]; Bruaroey, 2019).

Dojde-li během hovoru ke změnám spojení (změní se stopy, které klienti posílají, či se změní pozice klienta na síti), proběhne tzv. *renegotiation*, kdy se proces výměny session descriptions opakuje (Bruaroey, 2019).

SDP se skládá z párů klíč-hodnota (*key-value pairs*). Každý pár má následující formát: <klíč>=<hodnota>. SDP může stejný klíč obsahovat několikrát. (Bound; DuBois; voluntas, [b.r.]). Je-li hodnota nedefinovaná, používá se znak - (Begen; Kyzivat; Perkins; Handley, 2021). Záleží na pořadí jednotlivých řádků – SDP začíná obecnými informacemi o spojení, které jsou následovány jednotlivými média-sekcemi. Média-sekce popisuje jednu odchozí stopu a jednu příchozí stopu (obě stopy nemusí být nutně aktivní), které musí být stejného typu (audio, anebo video). SDP podporuje několik klíčů, WebRTC využívá následující (Bound; DuBois; voluntas, [b.r.]; Begen; Kyzivat; Perkins; Handley, 2021):

- **v** – značí verzi, která je vždy 0
- **o** – popisuje tvůrce spojení, slouží jako jeho unikátní identifikátor, skládá se z:
 - **username** – ID uživatele, který zahájil spojení
 - **sess-id** – unikátní číslo, které identifikuje spojení
 - **sess-version** – verze session description, která by se měla po každé změně zvýšit
 - **nettype** – typ sítě, je vždy IN (internet)
 - **addrtype** – typ adresy, je buď IP4, nebo IP6
 - **unicast-address** – IP adresa tvůrce spojení
- **s** – název session, SDP nesmí obsahovat více než jeden klíč **s**
- **c** – informace o spojení, pro WebRTC není klíč **c** příliš zásadní, neboť se pro spojování využívá protokolu ICE (více v části 2.1), skládá se z:
 - **nettype** – typ sítě, je vždy IN (internet)
 - **addrtype** – typ adresy, je buď IP4, nebo IP6
 - **connection-address** – IP adresa využita ke spojení
- klíč **t** – čas, kdy spojení začíná a končí, skládá se z:

- `start-time` – čas, kdy spojení začne; pokud je 0, spojení je permanentní
- `stop-time` – čas, kdy spojení skončí; pokud je 0, doba spojení není omezena
- `a` – atribut, jeho formát je `a=<jméno>:<hodnota>`; **SDP** může obsahovat několik klíčů `a`, ať už pro celé spojení, nebo pro specifickou média-sekci
- `m` – popis médií, skládá se z:
 - `media` – typ média, je buď `audio`, `video`, `application`, `text`, nebo `message`
 - `port` – port, který je využit k posílání dat
 - `proto` – protokol, hodnoty mohou být:
 - * `udp` – data jsou posílána přímo přes protokol **UDP**
 - * `RTP/AVP` – data jsou posílána pomocí protokolu RTP (více v části 2.2.1) s profilem pro audio a video
 - * `RTP/SAVP` – data jsou posílána pomocí protokolu **SRTP** (více v části 2.4.2)
 - * `RTP/SAVPF` – data jsou posílána pomocí protokolu **SRTP** s podporou pro **RTCP** (více v části 2.2.2)
 - `fmt` – seznam čísel, která určují podporované formáty dat; čísla od 0 do 95 referují ke specifickým formátům; u čísel od 96 do 127 se využívá atributu `a=rtpmap:<fmt> <codec>` pro specifikaci kodeku

SDP podporuje několik různých atributů, ty následující jsou zásadní pro fungování **WebRTC** (Bound; DuBois; voluntas, [b.r.]; Begen; Kyzivat; Perkins; Handley, 2021; Schulzrinne; Camarillo, 2010; Holmberg; Shpount; Loreto; Camarillo, 2021):

- `a=group:BUNDLE` značí, že přes jedno peer spojení posíláme více stop; některé implementace vytváří jedno peer spojení na každou stopu, což ale není doporučeno
- `a=sendrecv`, `a=sendonly`, `a=recvonly` a `a=inactive` – značí, směr dat dané média-sekce; oba klienti by se měli shodnout na směru dat, obsahuje-li tedy **SDP** offer `a=sendonly`, **SDP** answer by měla obsahovat `a=recvonly` pro danou média-sekci
- `a=mid` – číslo média-sekce
- `a=msid` – ID stopy a streamu, má formát `<ID streamu> <ID stopy>`
- `a=rtpmap` – zmíněno v předchozím odstavci
- `a=sctp-port` – port používaný protokolem **SCTP** (více v části 2.3)

- `a=max-message-size` – maximální velikost zprávy v bytech

SDP zároveň podporuje několik atributů, které se využívají pro nastavení **ICE** a výměnu **ICE** kandidátů (Bound; DuBois; voluntas, [b.r.]; Petit-Huguenin; Nandakumar; Holmberg; Keränen; Shpount, 2021):

- `a=ice-ufrag` – uživatelské jméno, používáno pro ověření integrity
- `a=ice-pwd` – heslo, používáno pro ověření integrity
- `a=ice-options` – specifikuje rozšíření, které klient podporuje
- `a=candidate` – je na úrovni média-sekce a specifikuje informace o daném **ICE** kandidátu

Níže můžeme vidět příklad reálného **SDP**, které obsahuje 3 média-sekce. Obě používají port 4000 a protokol RTP/AVP. V klíči `o` je důležité třetí pole s hodnotou 2, která indikuje verzi session description. Klíč `s` nemá hodnotu, spojení je tedy bez názvu. Klíč `c` je pak pro **WebRTC** nepodstatný.

```
v=0
o=- 2021867665009852994 2 IN IP4 127.0.0.1
s=-
c=IN IP4 192.168.1.1
t=0 0
a=group:BUNDLE 0 1
a=ice-ufrag:o6S/
a=ice-pwd:mXxyUi9PVAiuCPv1
a=ice-options:trickle
m=audio 4000 RTP/AVP 0 111
a=mid:0
a=sendrecv
a=candidate:foundation 1 udp 2130706431 192.168.1.1 53165 typ host generation 0
a=end-of-candidates
a=msid:yvKPspshcYcwGFTw DfQnKjQQuweLFdV
a=rtpmap:111 OPUS/48000/2
m=video 4000 RTP/AVP 96
a=mid:1
a=sendonly
a=msid:yvKPspshcYcwGFTw Ii65zvjkITaFf0T
a=rtpmap:96 VP8/90000
m=application 9 UDP/DTLS/SCTP webrtc-datachannel
a=mid:2
a=sctp-port:5000
a=max-message-size:262144
```

První sekce je **audio**, její **mid** je 0 a její směr je **sendrecv**, takže umožňuje přijímání i posílání audio stopy. Formát je dynamický a z **a=rtpmap** je vidět, že 111 označuje kodek Opus. Můžeme si rovněž povšimnout specifikace ID streamu a stopy.

Druhá sekce je **video**, její **mid** je 1 a její směr je **sendonly**, klient tedy může používat tuto média-sekci pro posílání videa. Formát je opět dynamický, z **a=rtpmap** je vidět, že číslo 96 označuje kodek VP8. Je také vidět, že stopa patří do stejného streamu jako stopa v první média-sekci.

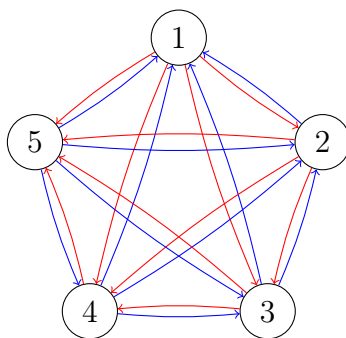
Třetí sekce je **application** a popisuje datový kanál. Její **mid** je 2. Protokol **SCTP** využívá port 5000 a maximální velikost zprávy je 262144 B, což je 256 KiB.

Další klíče se týkají protokolu **ICE**, který je popsán v části 2.1.4.

3 Skupinové volání pomocí WebRTC

Jak bylo zmíněno v části 2, protokol **WebRTC** funguje na **P2P** bázi (Google, [b.r.(a)]). To přináší otázku, zda a jak využít tuto technologii pro skupinové volání – hovory, které mají více než dva účastníky. K řešení tohoto problému existuje několik přístupů, které si následně rozebereme.

3.1 Full-mesh



Obrázek 5: Full-mesh

Prvním způsobem je tzv. spojení *full-mesh*, v rámci kterého každý klient komunikuje s každým dalším klientem, jak je zobrazeno na obrázku 5. Hlavní výhodou tohoto přístupu je, že nevyžaduje žádné *backend komponenty* (servery) – není tedy třeba vývojářů, kteří by na backendu pracovali, a zároveň není nutné vkládat finanční prostředky do jeho provozu. Nehrozí ani, že při selhání backendu by se uživatelé klientů nemohli spojit. Tento přístup má ale bohužel také mnoho nevýhod (Grozev, 2019).

První nevýhodou může být složitější vytvoření samotného spojení dvou klientů – v klasickém **P2P** modelu obvykle dochází k tomu, že jeden uživatel volá druhého, to samé ale u full-mesh spojení platit nemusí. Když se nový uživatel pokouší připojit do skupinového hovoru, je nejasné, zda jeho klient volá klienty, kteří se hovoru již účastní, či naopak. Je sice možné definovat příslušné pravidlo, ale v případě, že se dva uživatelé pokusí připojit ve stejný okamžik, je situace komplikovanější, neboť nelze jednoznačně určit, kdo je volající a kdo volaný. Jedním řešením jsou tzv. *algoritmy pro glare resolution*, které řeší situaci, kdy se klienti snaží kontaktovat navzájem ve stejný okamžik. Obvykle každý klient vygeneruje náhodné číslo, které je poté v případě, že dochází ke *glare* situaci, porovnáno s číslem druhého klienta a klient s větším či menším číslem (záleží na implementaci) je pak vybrán jako volající (Westerlund, 2011). Někdy také dochází k lexikografickému porovnání identifikátorů klientů pro zvolení klienta, který bude kontaktovat toho druhého (Hodgson; Long; Townsend; Brandner, [b.r.]).

Full-mesh spojení má také nevýhodu v tom ohledu, že může častěji docházet k tzv. *split-brains* (Haddock, 2010). Protože je každý klient spojen s každým dalším klientem, je spojení velmi náchylné k chybám, hrozí tedy, že jeden z klientů bude přijímat streamy jen od určité podskupiny klientů, zatímco jiný klient bude přijímat stream od jiné podskupiny klientů, a tak dva uživatelé, kteří se hovoru účastní, můžou mít značně odlišný zážitek.

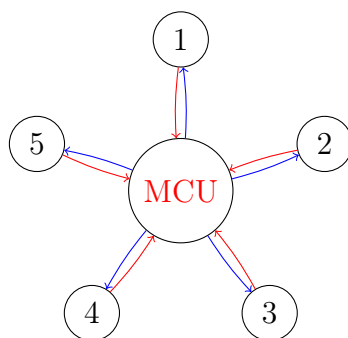
Dalším problémem je zátěž klienta, protože musí přijímat **RTP** streamy všech klientů, které nejdříve rozšifruje a poté rozkóduje. Zároveň posílá všechny své **RTP** streamy ostatním klientům, ty musí naopak zakódovat a následně zašifrovat. Při vyšším počtu klientů v hovoru je pak třeba, aby všichni klienti běželi na dostatečně silném hardwaru (Kesby, [b.r.]; Infrared5, 2021).

Další bariérou je internetové připojení – většina klientů totiž nemá dostatečně rychlou síť, aby mohli posílat **RTP** streamy všem uživatelům, a tak je realisticky možné pomocí full-mesh spojení provádět jen hovory v nižších počtech klientů (obvykle je maximum 5 - 8 klientů) (Levent-Levi, 2021).

3.2 Servery – foci

Možným řešením problémů s full-mesh spojením je využití backendu. V tomto případě bude každý klient posílat své **RTP** streamy serveru, ze kterého se tedy stává tzv. *focus* (bod, ve kterém jsou jednotlivé streamy soustředěny; množné číslo je *foci*). Tento focus poté streamy přeposílá ostatním klientům. Existují zde dva hlavní přístupy (Infrared5, 2021), které si níže rozebereme.

3.2.1 Multipoint Control Unit



Obrázek 6: Multipoint Control Unit

První možností je tzv. **Multipoint Control Unit (MCU)**, tento server přijímá streamy jednotlivých klientů a následně je transkóduje a mixuje do jednoho, který pak zpět posílá

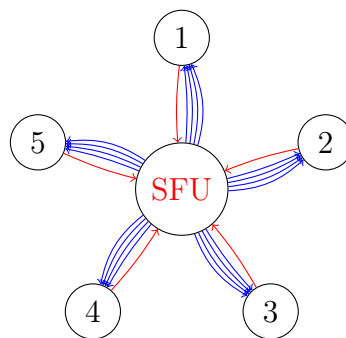
všem uživatelům (viz obrázek č. 6). Hlavní výhodou tohoto přístupu je, že klient nemusí mít vysokou rychlost stahování dat, neboť přijímá jen jeden stream. Má ale relativně dost nevýhod (Infrared5, 2021).

První z nich je, že streamy jsou mixované do jednoho streamu, který je pak pro všechny klienty stejný, neumožňuje tedy uživatelům klientů, aby si vybrali, který stream budou sledovat atp. (Infrared5, 2021).

Zároveň proces transkódování a mixování je poměrně náročný na výkon, příslušný server tedy potřebuje silný hardware. Tento proces je také mírně časově náročný, což znamená, že uživatelé vidí a slyší věci s mírným zpožděním (Infrared5, 2021).

Poslední nevýhodou, kterou zmíníme, je to, že pro transkódování a mixování streamů je nutné, aby je server nejdříve rozšířoval. To znamená, že není možné, aby komunikace mezi klienty byla E2EE (Abrell, 2020).

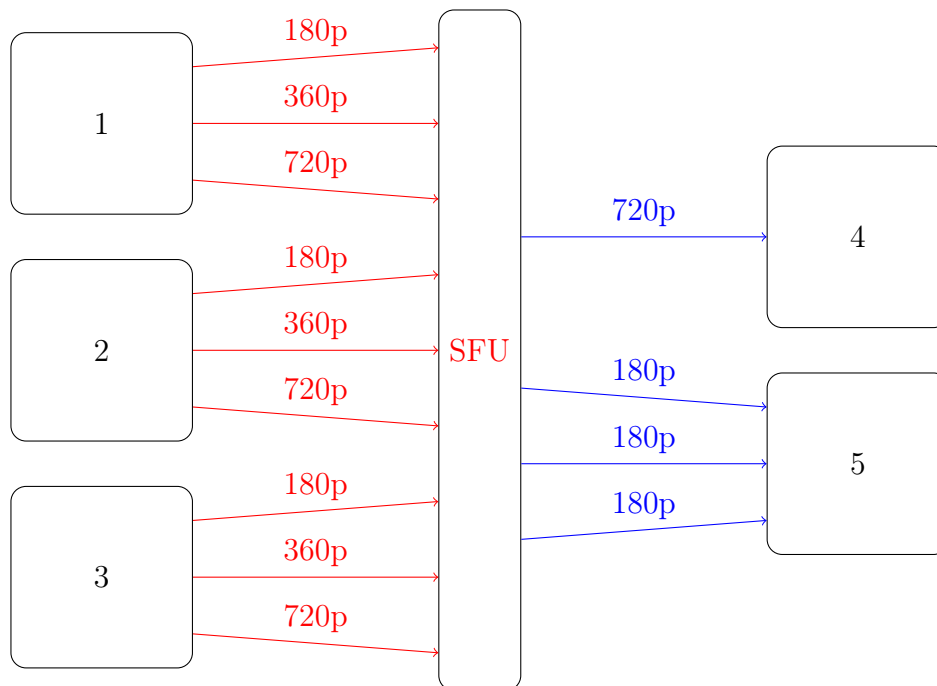
3.2.2 Selective Forwarding Unit



Obrázek 7: Selective Control Unit

Druhou možností je tzv. **Selective Forwarding Unit (SFU)**, tento server přijímá streamy jednotlivých klientů a následně podskupinu těchto streamů přeposílá ostatním klientům (Grozev, 2019), jak je vidět na obrázku č. 7.

Toto řešení má několik výhod – umožňuje E2EE, neboť nepotřebuje streamy transkódovat a mixovat (Hancke, 2020). Z tohoto důvodu také nemá takové požadavky na hardware, jednotlivé streamy jsou klientům posílány samostatně, takže si uživatel může zvolit, co bude sledovat (Infrared5, 2021). Zároveň tento model umožňuje tzv. *federaci* (více v částech 5 a 6).



Obrázek 8: Simulcast

Oproti **MCU** sice vyžaduje větší download ze strany klientů, neboť přijímají streamy jednotlivě. Toto ale není tak velký problém, neboť **SFU** může posílat jen určitou podskupinu streamů. Zároveň existuje metoda s názvem *simulcast*, kdy klient **SFU** posílá několik verzí svých streamů v různých rozlišeních (obvykle jsou to původní, poloviční a čtvrtinový) a s různým počtem snímků za sekundu, **SFU** poté přeposílá vhodnou verzi streamu (viz 8). Vhodná verze se obvykle vybírá na základě několika kritérií. Prvním z nich často je, v jakém rozlišení bude video zobrazeno na koncovém klientovi – např. jestli má uživatel jedno video velkém rozlišení a ostatní jsou malá, nebo či má všechny videa zobrazena ve stejném rozlišení bez toho, aby nějaké bylo zvětšené. Dalším faktorem při výběru je spojení s jednotlivým klientem a jeho kvalita (Zhao, 2021).

4 Další VoIP technologie

MatrixRTC a WebRTC samozřejmě nejsou jediné VoIP technologie – v této části se na tyto VoIP technologie podíváme.

4.1 Protokol SIP

Session Initiation Protocol (SIP) je protokol, který specifikuje komunikaci se signalling serverem, umožňuje oznámit ostatním klientům, že je klient aktivní, volat ostatní klienty, ukončení hovoru atp (Fry, 2021).

4.2 LiveKit

LiveKit je open-source projekt, který by měl zjednodušit práci vývojářů s WebRTC. Jedná se o SFU (viz 3) a kolekci klient Software Development Kit (SDK) (LiveKit, [b.r.(a)]; LiveKit, [b.r.(b)]).

Jeho SFU využívá Pion a jako vrstva pro signalling se využívá WebSockets (LiveKit, [b.r.(b)]).

4.3 Protokol aplikace Zoom

Zoom je konferenční software, který stojí na protokolech SIP, Real Time Messaging Protocol (RTMP) a WebRTC. SIP se využívá na signalling, RTMP na samotný přenos médií a WebRTC API pro získání streamu dat z kamery či obrazovky. Ve webových prohlížečích používá pro přenos médií WebSockets, což je zvláštní rozhodnutí vzhledem k tomu, že WebSockets používají TCP (Hancke, 2019; Wallis, 2023; Zoom Video Communications, [b.r.]).

4.4 Protokol aplikace Discord

Discord je aplikace na chat a VoIP hovory (Discord, [b.r.]). Používá protokol WebRTC jako základ své technologie, neboť je podporován na celé škále prohlížečů a operačních systémů. V prohlížečích Discord využívá jejich nativní implementace WebRTC, ale na desktopu, Androidu a iOS využívají nativní knihovnu pro WebRTC napsanou v C++; některé funkce tedy fungují lépe např. v desktopové aplikaci než v prohlížeči (Vass, 2018).

Nativní knihovna v C++ umožňuje Discordu větší kontrolu nad WebRTC, což dává prostor pro různá zjednodušení. Vzhledem k tomu, že v případě Discordu klient vždy volá server, je možné vynechat ICE. Discord také používá Salsa20 šifrování, protože je

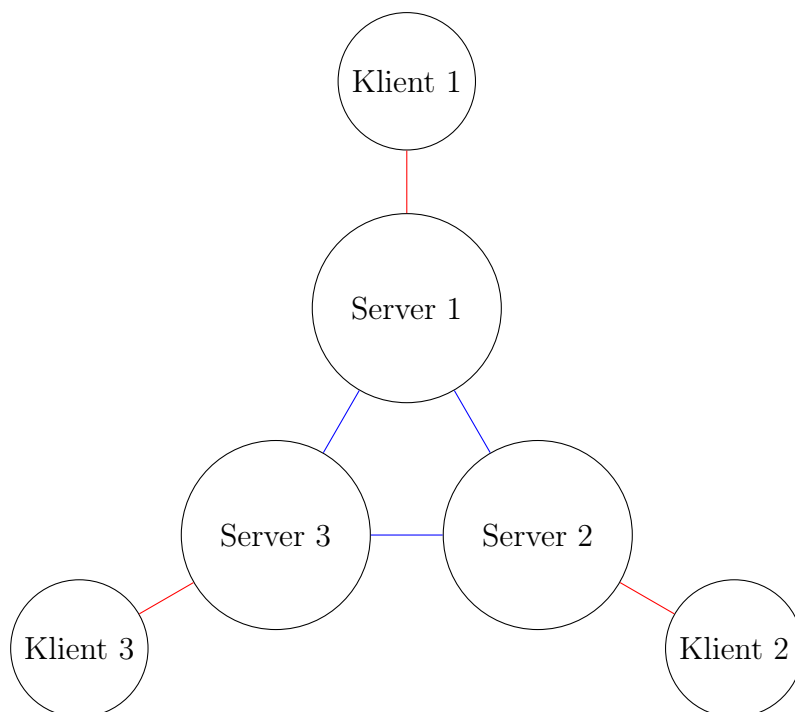
rychlejší než **DTLS** a **SRTP**. Zároveň šetří data tím, že neposílá žádné pakety, pokud uživatel zrovna nemluví (Vass, 2018).

Signalling server používá WebSockets a je napsaný v jazyce Elixir (Vass, 2018).

4.5 Protokol aplikace Jitsi

Jitsi je kolekce open-source projektů, které poskytují **VoIP** služby, které jsou vhodné pro self-hosting (8x8, [b.r.(a)]). Jako signalling vrstvu používá protokol XMPP (XMPP Standards Foundation, [b.r.(b)]).

5 Matrix



Obrázek 9: Matrix federace

Protokol Matrix je otevřený standard pro bezpečnou, decentralizovanou komunikaci v reálném čase. Protokol byl vytvořen jako způsob řešení problému tzv. *walled gardens*. Sám o sobě je protokol federovaný, tedy podobně jako email umožňuje klientům jednoho serveru komunikovat s klienty serveru druhého (viz obrázek č. 9). V protokolu Matrix se těmito serverům říká *homeservery*. Od června 2019 je stabilní (The Matrix.org Foundation, [b.r.(b)]); The Matrix.org Foundation, [b.r.(c)]).

Je spravován neziskovou organizací jménem Matrix.org Foundation a jeho referenční implementace používají permissivní⁵ licenci Apache (The Matrix.org Foundation, [b.r.(c)]).

5.1 Specifikace a její změny

Jak **stabilní specifikace**, která je vydávána jednou za čtvrt roku, tak **nestabilní specifikace** jsou volně dostupné a kdokoli je může implementovat.

Vzhledem k tomu, že protokol je otevřený a umožňuje všem navrhopvat změny, existuje proces pro návrh a schvalování těchto změn. Chce-li někdo navrhnout změnu

⁵Licence, která nevyžaduje od odvozených děl, aby využívali stejnou licenci (Myška; Polčák; Šavelka; Kyncl; Sviráková, 2014).

protokolu, může napsat tzv. **Matrix Spec Change (MSC)** návrh. Jedná se o pull request⁶ navrhuující úpravu repositáře **matrix-spec-proposals**. Tento pull request obsahuje text formátovaný pomocí **GitHub-flavored Markdownu**⁷ a popisuje problém, který se **MSC** snaží vyřešit, možnosti řešení a výhody a nevýhody daných přístupů k řešení (The Matrix.org Foundation, [b.r.(f)]).

Poté, co je pull request vytvořen, nastává prostor pro to, aby komunita a vývojáři **MSC** komentovali a aby se případné připomínky řešily. **MSC** obvykle také bude zkontrolováno členy **Spec Core Team (SCT)**, který se o specifikaci stará. Pokud se **MSC** dostane do stavu, kdy nejsou vznášeny připomínky (a ty, co byly vznesené, byly vyřešeny), může člen **SCT** navrhnout spuštění tzv. **Final Comment Period (FCP)**. Pro to, aby **FCP** začala, je třeba souhlas 75 % **SCT** (The Matrix.org Foundation, [b.r.(f)]).

Když **FCP** započne, je prostor 5 dnů na to, aby kdokoliv vznesl připomínky. Jsou-li vzneseny závažné připomínky, člen **SCT** může **FCP** zablokovat. Proběhne-li **FCP** do konce, je **MSC** schváleno a nastává čas pro samotnou změnu specifikace, jedná se obvykle další pull request, který vytvoří člen **SCT** a navrhuje změny repositáře **matrix-spec**. Když je tento pull request přijat, změna se stala součástí nestabilní verze specifikace a bude v další stabilní verzi (The Matrix.org Foundation, [b.r.(f)]).

5.2 Primitiva

Matrix má několik základních primitiv, které si zde rozebereme.

5.2.1 Uživatelé

Každý klient je spojený s uživatelským účtem, ty jsou identifikované pomocí **user_id**. To má obecně následující formu: **@localpart:domain**. Reálné uživatelské ID pak může vypadat takto: **@alice:server.org** (The Matrix.org Foundation, [b.r.(e)]).

5.2.2 Zařízení

Zařízení mají v protokolu Matrix specifický význam – nejedná se o fyzická zařízení, ale jedná se o abstrakce jednotlivých přihlášených klientů. Každý uživatel může mít několik zařízení, ze kterých využívá svůj účet. Zařízení je obvykle identifikováno pomocí **user_id** a **device_id** (The Matrix.org Foundation, [b.r.(e)]).

Zařízení mají hlavně význam pro šifrování – viz část 5.6.

⁶Pull request je způsob pro návrhu změn zdrojového kódu, diskuzi o nich a případné následné přijetí těchto změn (GitHub, [b.r.(a)]).

⁷Markdown je jednoduchý mark-up jazyk pro formátování textu (Gruber, [b.r.]). GitHub-flavored Markdown je pak Markdown, který používá GitHub (GitHub, [b.r.(b)]).

5.2.3 Místnosti a události

Velice zásadním primitivem v protokolu Matrix je *místnost* (*room*). V základní podobě je Matrix místnost reprezentací místnosti pro chat, ale má mnoho jiných využití, neboť se více méně jedná o decentralizovanou databázi pro JSON data. Místnost ani její obsah neexistuje na žádném jediném homeserveru – každý homeserver má její kopii, kterou udržuje synchronizovanou s ostatními homeservery. Kromě místností pro chat, existují např. tzv. *prostory* (*spaces*), které odkazují na jiné místnosti, a tak umožňují vytvořit hierarchickou strukturu místností (analogií jsou např. týmy v aplikaci MS Teams) (The Matrix.org Foundation, [b.r.(e)]).

Místnost obsahuje tzv. *události* (*events*), které jsou obvykle výsledkem akce na straně klienta (např. posílání zprávy). Události se dělí na dva druhy – prvním druhem jsou *message události*, jedná se např. o běžné zprávy a reakce, druhým typem jsou *state události*, které reprezentují trvalé informace, jako je např. jméno místnosti, téma místnosti, členství v místnosti atp. (The Matrix.org Foundation, [b.r.(e)]).

Vzhledem k tomu, že state události můžou být měněny více uživateli, existuje tzv. *state resolution algorithm*, který umožňuje homeserverům se shodnout na obsahu state událostí, pokud došlo k tomu, že homeservery nebyly nějakou dobu ve spojení (The Matrix.org Foundation, [b.r.(e)]).

5.2.4 To-device messages

To-device messages jsou dalším druhem událostí, který ale neexistuje v místnosti, neboť je přeposílán z jednoho zařízení do druhého. Aktuálně má hlavně dvě využití – šifrování (více v části 5.6) a skupinové volání (více v části 6).

5.3 Klienti

V tento moment existují pravděpodobně stovky Matrix SDK a klientů, z nichž je většina vyvíjena komunitou. Mezi ty známé patří např. Element (Element, [b.r.(a)]), Hydrogen (Element, [b.r.(h)]), Nheko (Nheko-Reborn, [b.r.]), FluffyChat (Fear, [b.r.]) či Cinny (Bura, [b.r.]).

Element je všeobecný klient pro Matrix chat (Element, [b.r.(a)]), jeho webová verze používá SDK `matrix-js-sdk`, které je napsané v TypeScriptu (matrix.org, [b.r.(b)]). Stejně SDK používá klient Cinny, který je specifický svým uživatelským rozhraním inspirovaným Discordem (Bura, [b.r.]; Cinny, [b.r.]). Jedná se o SDK, které je jednou z referenčních implementací protokolu Matrix (matrix.org, [b.r.(b)]). Desktopová verze Elementu je pak

Electron⁸ wrapper jeho webové verze (Element, [b.r.(d)]). Hydrogen je rovněž webová aplikace, používající své vlastní SDK, které je z části v TypeScriptu a z části v JavaScriptu. Toto SDK ale využívají i jiní klienti (viz 8.1.1). Hydrogen má za cíl vyžadovat co nejnižší požadavky na hardware a měl by běžet i v prohlížečích, jako je Internet Explorer⁹ (Element, [b.r.(h)]).

Android a iOS verze Elementu využívají každá své vlastní SDK (v Kotlinu a Objective C) (*Element Android*, [b.r.]; Element, [b.r.(e)]), probíhá však vývoj nástupců těchto aplikací jménem Element X, které budou využívat *matrix-rust-sdk* (Element, [b.r.(f)]; Element, [b.r.(g)]).

Nheko je desktopový klient napsaný v jazyce C++ s pomocí knihovny Qt pro tvoření uživatelských rozhraní (Nheko-Reborn, [b.r.]).

FluffyChat je klient zaměřený na jednoduchost používání, je napsaný v jazyce Dart, a tak běží na operačních systémech Android a iOS, ale i na webu a desktopu (Fear, [b.r.]; Famedly, [b.r.]).

5.4 Homeservery

Tři nejvýznamnější implementace homeserverů jsou *Synapse*, který je napsaný v jazyce Python, jedná se o historicky první a referenční implementaci homeserveru (matrix.org, [b.r.(e)]). Druhou je *Dendrite*, která je napsaná v jazyce Go a má být nástupce Synapse (matrix.org, [b.r.(a)]). Poslední implementací je *Conduit*, která je napsaná v jazyce Rust a je spravovaná komunitou (Koesters, [b.r.]).

Mezi další homeservery patří např. *Construct* v jazyce C++ (Matrix Construct, [b.r.]).

5.5 Přemostění (bridging)

Vzhledem k malé pravděpodobnosti, že by společnosti jako Google, či Microsoft začaly používat protokol Matrix, umožňuje tento protokol tzv. *bridging*. Jedná se o způsob, jak „přemostit“ uživatele z jiných platforem (jako je např. Messenger či MS Teams) do protokolu Matrix (The Matrix.org Foundation, [b.r.(d)]).

5.6 End-to-end šifrování

Protokol Matrix podporuje **E2EE** – message události v místnostech mohou být šifrované. Pro dosažení **E2EE** používá jak symetrické, tak asymetrické šifrování. Jednotlivé

⁸Electron je framework pro vytváření desktopových aplikací pomocí webových technologií (OpenJS Foundation and Electron contributors, 2020).

⁹Internet Explorer je jeden z nejvíce nenáviděných internetových prohlížečů, jedním z důvodů je, že nepodporuje mnoho moderní funkcí, ale mnoho vývojářů je s ním přesto nuceno pracovat (Zeal, [b.r.]).

message události jsou zašifrovány pomocí symetrického šifrování **Advanced Encryption Standard (AES)**. K přeposílání klíčů mezi zařízeními, které jsou v dané místnosti, se používají to-device messages šifrované pomocí asymetrické kryptografie nad eliptickými křivkami (využívají se především dvě knihovny – **olm**, která je napsaná v jazyce C (matrix.org, [b.r.]), a **vodozamac**, která je napsaná v jazyce Rust a je součástí **matrix-rust-sdk** (matrix.org, [b.r.(d)]).

Aktuálně není možné šifrovat state události, ale existují **MSC**, které by to umožnily (Ralston, [b.r.]). Zároveň se pracuje na integraci protokolu **Messaging Layer Security (MLS)**, který má fungovat lépe v místnostech s velkým množstvím zařízení. **MLS** je ale třeba rozšířit tak, aby podporovalo federované systémy, rozšíření se pak nazývá **Decentralized Messaging Layer Security (DMLS)** (The Matrix.org Foundation, [b.r.(a)]).

6 MatrixRTC

Stabilní verze protokolu Matrix podporuje hlasové a video hovory mezi dvěma uživateli, ale pouze v základní podobě. Skupinové volání, pozastavování hovorů, renegotiation, algoritmy pro glare resolution, atp. ale aktuálně nejsou součástí stabilní verze protokolu (The Matrix.org Foundation, [b.r.(e)]).

Kvůli tomu existuje několik MSC, které navrhuji změny specifikace pro přidání nových funkcí. MSC2746 přidává podporu pro renegotiation, pozastavování hovorů, upgrady hovorů, restartování ICE, lepší podpora pro více zařízení a Dual-Tone Multi-Frequency (DTMF) (Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [b.r.]). MSC3077 přidává podporu pro hovory s více streamy (např. sdílení obrazovek) (Brandner; Asokan, [b.r.]). MSC3291 pak specifikuje sdílení metadat o streamech (např. zda je audio či video ztlumené) (Brandner; Morgan, [b.r.]).

MSC3401 přidává podporu pro skupinové full-mesh hovory (Hodgson; Long; Townsend; Brandner, [b.r.]). MSC3898 pak přidává podporu pro foci (množné číslo od *focus*) (Hodgson; Brandner; Abramov; Baker, [b.r.]).

6.1 Využití Matrix primitiv

Podpora pro VoIP hovory v protokolu Matrix je postavena na již existujících primitivech. V této části probereme, jak se jednotlivé primitiva využívají.

6.1.1 Hovory mezi dvěma uživateli

Hovory mezi dvěma uživateli (stabilní specifikace, MSC2746, MSC3077, MSC3291) využívají message událostí (The Matrix.org Foundation, [b.r.(e)]; Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [b.r.]; Brandner; Asokan, [b.r.]; Brandner; Morgan, [b.r.]).

Typy všech VoIP událostí mají předponu `m.call.` a jejich `content` obsahuje `call_id` (unikátní identifikátor pro daný hovor), `party_id` (unikátní identifikátor pro účastníka hovoru; může být `device_id`), `version` (verze VoIP specifikace; měla by být "1") a `lifetime` (čas v ms, po který je událost validní) (The Matrix.org Foundation, [b.r.(e)]; Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [b.r.]).

Pro zahájení hovoru se využívá událost s typem `m.call.invite`. Její `content` obsahuje `offer`, která obsahuje `sdp` (SDP offer posílajícího klienta) a `type` (který by měl vždy být "offer") (The Matrix.org Foundation, [b.r.(e)]; Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [b.r.]).

```
{
  "type": "m.call.invite",
  "content": {
    "version": "1",
    "call_id": "12345",
    "party_id": "first_party_id",
    "lifetime": 60000,
    "offer": {
      "sdp": "the SDP offer string",
      "type": "offer"
    }
  }
}
```

Poté, co druhý klient přijme tuto zprávu, by měl poslat na základě rozhodnutí uživatele buď `m.call.answer`, nebo `m.call.reject` (The Matrix.org Foundation, [\[b.r.\(e\)\]](#); Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [\[b.r.\]](#)).

Pokud se uživatel rozhodne hovor přijmout, jeho klient by měl poslat `m.call.answer`, jejíž `content` obsahuje `answer`, která obsahuje `sdp` (SDP answer) a `type` (který by měl vždy být `answer`) (The Matrix.org Foundation, [\[b.r.\(e\)\]](#); Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [\[b.r.\]](#)).

```
{
  "type": "m.call.answer",
  "content": {
    "version": "1",
    "call_id": "12345",
    "party_id": "other_party_id",
    "lifetime": 60000,
    "answer": {
      "sdp": "the SDP answer string",
      "type": "answer"
    }
  }
}
```

Pro předání dalších ICE kandidátů druhému klientovi je použita událost s typem `m.call.candidates`, která obsahuje `candidates`, což je seznam ICE kandidátů (The Matrix.org Foundation, [\[b.r.\(e\)\]](#)). Například JavaScript API pro WebRTC umožňuje kandidáty konvertovat do JSON pomocí `RTCIceCandidate::toJSON()` (MDN contributors, [\[b.r.\(a\)\]](#)).

```

{
  "type": "m.call.candidates",
  "content": {
    "version": "1",
    "call_id": "12345",
    "party_id": "1355613132",
    "candidates": [
      {
        "candidate": "the candidate string",
        "sdpMLLineIndex": 0,
        "sdpMid": "audio"
      }
    ]
  }
}

```

Pokud se uživatel rozhodne hovor odmítnout, jeho klient by měl poslat `m.call.reject` (Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [\[b.r.\]](#)).

```

{
  "type": "m.call.reject",
  "content": {
    "version": "1",
    "call_id": "12345",
    "party_id": "1355613132"
  }
}

```

Je-li při hovoru třeba provést renegotiation, klient by měl poslat událost s typem `m.call.negotiate`, jejíž `content` by měl obsahovat `description` s `sdp` (`SDP` offer nebo answer) a `type` ("offer" nebo "answer") (Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [\[b.r.\]](#)).


```
{
  "type": "m.call.negotiate",
  "content": {
    "version": "1",
    "call_id": "12345",
    "party_id": "1355613132",
    "lifetime": 10000,
    "description": {
      "sdp": "the SDP string",
      "type": "offer"
    }
  }
}
```

Pokud je hovor ukončen, klient by měl poslat událost s typem `m.call.hangup`, jehož `content` obsahuje pole `reason`, které nabývá několika hodnot (Baker; Hodgson; DeepBlueV7.X; Brandner; Chathi; Hoff, [b.r.]):

- `user_hangup` – uživatel ukončil hovor
- `invite_timeout` – druhá strana neodpověděla včas
- `ice_failed` – **ICE** negotiation selhala
- `ice_timeout` – spojení selhalo poté, co už byla přeposlána nějaká data
- `user_media_failed` – klientovi se nepodařilo získat přístup ke streamu z příslušného zařízení
- `user_busy` – primárně pro přemostěné **Public Switched Telephone Network (PSTN)** hovory, kde druhá strana může být nedostupná, protože už s někým telefonuje
- `unknown_error` – došlo k neznámé chybě

```
{
  "type": "m.call.hangup",
  "content": {
    "version": "1",
    "call_id": "12345",
    "party_id": "1355613132",
    "reason": "user_hangup"
  }
}
```

6.1.2 Skupinové hovory

Pro vytvoření skupinového hovoru se využívá `m.call` state událost, jejíž `content` obsahuje několik polí (Hodgson; Long; Townsend; Brandner, [b.r.]):

- `m.intent` – může nabývat několika hodnot `m.room` (otevře-li uživatel danou místnost, hovor by měl být jejím hlavním obsahem – podobně jako Discord voice a video channels), `m.prompt` (uživatel by měl být upozorněn, že hovor probíhá), `m.ring` (klient uživatele by měl začít vyzvánět)
- `m.type` – může být `m.voice`, nebo `m.video`
- `m.name` – libovolný název pro hovor

```
{
  "type": "m.call",
  "state_key": "cvsiu2893",
  "content": {
    "m.intent": "m.room",
    "m.type": "m.voice",
    "m.name": "Voice room"
  }
}
```

Chce-li se uživatel připojit do hovoru, pošle state událost typu `m.call.member`, jejíž `content` obsahuje informace o tom, které zařízení se účastní jakého hovoru (Hodgson; Long; Townsend; Brandner, [b.r.]). Neobsahuje-li událost `m.foci`, nebo se hovoru účastní jen dva klienti, dojde ke spojení full-mesh; chování pokud událost tento klíč obsahuje a v hovoru je více klientů je popsáno v části 6.3 (Hodgson; Brandner; Abramov; Baker, [b.r.]). Následně se mezi jednotlivými zařízeními využívá to-device messages, které jsou ve stejném formátu jako události popsané v části 6.1.1 (Hodgson; Long; Townsend; Brandner, [b.r.]).

```

{
  "type": "m.call",
  "state_key": "@user:server.org",
  "content": {
    "m.calls": [
      {
        "m.call_id": "cvsiu2893",
        "m.devices": [
          {
            "device_id": "ASDUHDGFYUW",
            "session_id": "GHKJFKLJLJ",
            "expires_ts": 1654616071686,
            "m.foci.active": [
              {
                "user_id": "@sf1:server.org",
                "device_id": "FS5F589EF"
              }
            ],
            "m.foci.preferred": [
              {
                "user_id": "@sfu2:server.org",
                "device_id": "3FSF589EF"
              },
              {
                "user_id": "@sfu3:server.org",
                "device_id": "GFSDH93EF"
              }
            ]
          }
        ]
      }
    ]
  }
}

```

6.2 Klientské SDK

Aktuálně neexistuje velké množství implementací MatrixRTC, hlavními ale jsou ty, které si popíšeme v následující sekci.

6.2.1 Matrix JavaScript SDK

[matrix-js-sdk](#) je aktuálně nejrozšířenější implementací MatrixRTC s podporou nejvíce funkcí. Podporuje renegotiation, pozastavování hovorů, upgrady hovorů, restartování

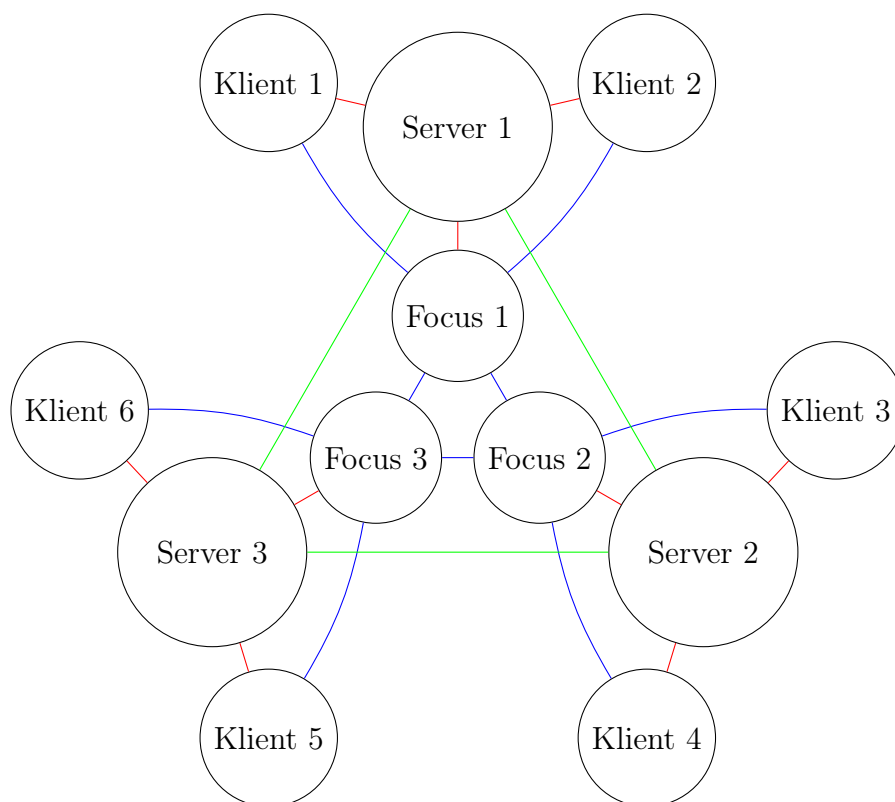
ICE, více zařízení, DTMF, více streamů, sdílení metadat o streamech a skupinové hovory (matrix.org, [b.r.(b)]). Zároveň se na experimentální větvi pracuje na podpoře MSC3898 (Hodgson; Brandner; Baker; Schwendig, [b.r.]).

Element toto SDK využívá na hovory mezi dvěma uživateli a Element Call na skupinové hovory (Element, [b.r.(c)]; Element, [b.r.(b)]; matrix.org, [b.r.(c)]).

6.2.2 Hydrogen SDK

Další implementací je Hydrogen SDK, které je součástí klient **Hydrogen**. Využívá ho právě klient Hydrogen a Third Room (viz 8.1.1) (Element, [b.r.(h)]).

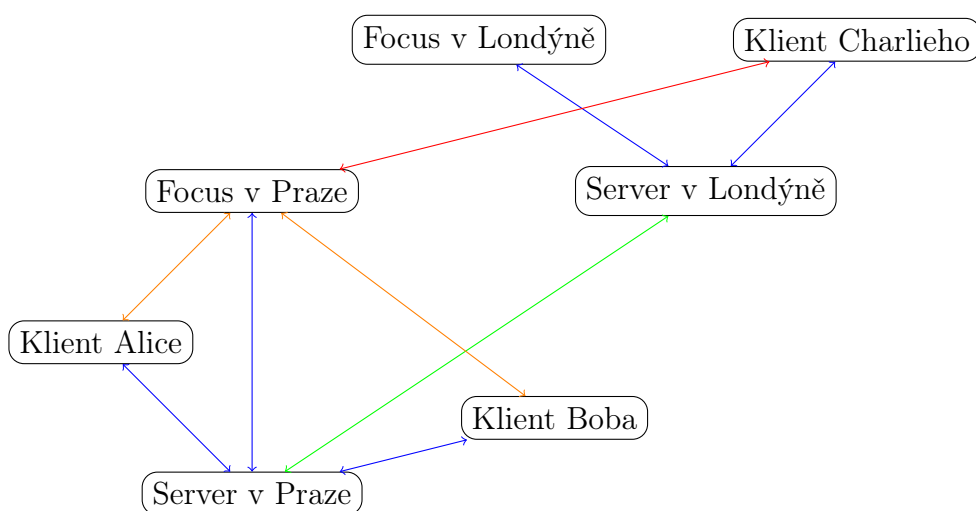
6.3 Foci



Obrázek 10: Federace Matrix foci

MSC3898 podporuje tzv. *federované foci*. Na obrázku 10 je vidět, kde jsou federované foci umístěné v Matrix topologii. Jedná se o **WebRTC** servery (může se jednat o **SFU**, **MCU** či jiný typ serveru), které jsou podobně jako samotné Matrix homeservery federovány (Hodgson; Brandner; Abramov; Baker, [b.r.]).

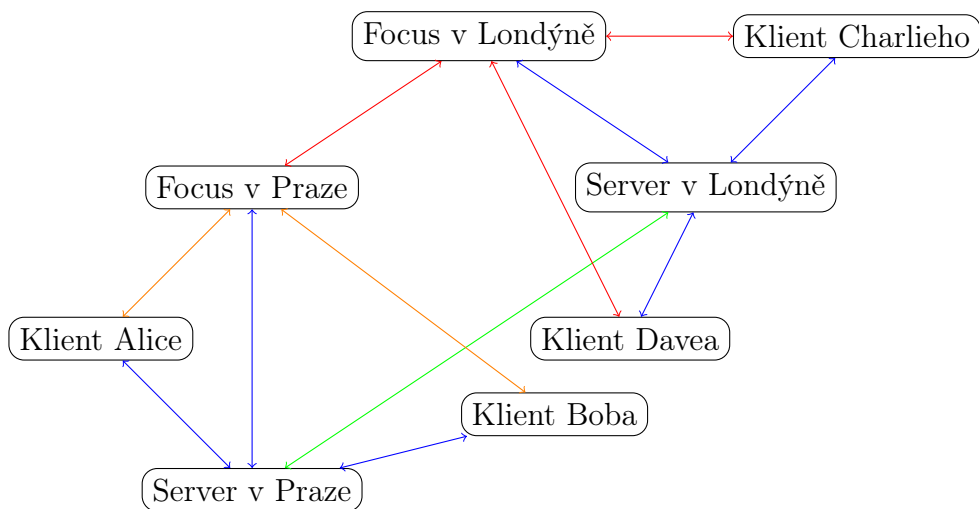
MSC3898 popisuje celý proces výběru a připojování k danému focusu (Hodgson; Brandner; Abramov; Baker, [b.r.]). My si situaci vysvětlíme na příkladu.



Obrázek 11: Federace Matrix foci: Charlie se připojuje do hovoru

Na obrázku 11 můžeme vidět situaci, kde máme dva Matrix homeservery (jeden v Praze a druhý v Londýně) a zároveň máme dva Matrix foci (opět jeden v Praze a druhý v Londýně). Alice a Bob používají pražský homeserver a jsou připojeni k hovoru, který se aktuálně odehrává na pražském focusu.

Do situace ale vstupuje nový uživatel Charlie, který používá homeserver v Londýně. Přestože by mohl použít londýnský focus, nepoužije ho a připojí se rovnou na ten pražský, neboť je aktuálně jediný účastník hovoru z Londýna (Hodgson; Brandner; Abramov; Baker, [b.r.]).



Obrázek 12: Federace Matrix foci: Dave se připojuje do hovoru

Na obrázku 12 vidíme, že se do hovoru připojil uživatel Dave, který je podobně jako Charlie v Londýně. Z důvodu, že už máme dva uživatele, kteří se nacházejí blíže londýnskému focusu, dojde k federaci obou foci (Hodgson; Brandner; Abramov; Baker, [b.r.]).

6.3.1 Waterfall

Waterfall je aktuálně jedinou implementací Matrix focusu. Je napsaná v jazyce Go pomocí knihovny Pion (matrix.org, [b.r.(f)]).

7 Implementace break-out rooms

Break-out rooms (dále break-out místnosti) je funkce mnoha konferenčních aplikací umožňující rozdělení velké konference do několika menších hovorů (Zoom Video Communications, 2023; Microsoft, [b.r.(c)]). Tato funkce sehrála důležitou roli např. ve školství během pandemie Covidu-19 (Ahmed; Khalid, 2021; Agustina; Ema; Suharya; Toto, 2021) a zůstává nadále relevantní.

Praktickou částí této mé práce je implementace break-out místností do klienta Element Call, který využívá protokol Matrix pro video-hovory. Než budeme ale samotné break-out místnosti implementovat, bude nutné navrhnout změnu Matrix specifikace, aby případně mohli i jiní klienti využívat tuto funkci.

7.1 Návrh na doplnění specifikace

Pro reprezentaci jednotlivých break-out místností využijeme již existující Matrix primitivum – místnost, každé break-out místnosti bude tedy odpovídat jedna Matrix místnost. Pro informování ostatních uživatelů o existenci daných break-out místností též využijeme existující Matrix primitivum – state události (Brandner, [b.r.(d)]):

```
{
  "type": "m.breakout",
  "state_key": "",
  "content": {
    "m.breakout": {
      "!roomId1": {
        "via": ["example.org", "other.example.org"],
        "users": ["userId1", "userId2"]
      },
      "!roomId2": {
        "via": ["example.org", "other.example.org"]
      }
    }
  }
}
```

Pokud chce klient vytvořit např. 2 break-out místnosti, vytvoří dané Matrix místnosti a následně pošle state událost typu `m.breakout`, jejíž příklad můžeme vidět výše. V poli `content` můžeme najít pole `m.breakout`, ve kterém klíče jsou ID Matrix break-out místností (`!roomId1` a `!roomId2`) a hodnoty obsahující informace o těchto místnostech. V informacích o místnosti najdeme pole `via`, což je výčet serverů, které je možno využít k připojení se k dané místnosti (ID místnosti nestačí pro připojení se k místnosti (The

Matrix.org Foundation, [b.r.(e)]). V těchto informacích můžeme též najít pole `users`, což je výčet uživatelů, kterým je doporučeno se do dané místnosti připojit (Brandner, [b.r.(d)]).

Dojde-li k přijetí této state události jiným klientem, měl by uživateli dát možnost se připojit do daných místností, zároveň by ve svém uživatelském rozhraní měl zobrazit, kteří uživatelé náleží kterým místnostem (Brandner, [b.r.(d)]).

Jelikož je nepraktické, aby v místnosti bylo víc událostí typu `m.breakout`, není to povoleno. Toho docílíme tím, že jako `state_key` budeme vždy využívat prázdný řetězec znaků (tedy `""`) (Brandner, [b.r.(d)]) – v Matrix místnosti vždy existuje jen jedna validní state událost s danou dvojicí `type` a `state_key` (The Matrix.org Foundation, [b.r.(e)]).

Na GitHubu v repositáři [matrix-spec-proposals](#) můžeme najít pull request č. 3985 – [MSC3985](#), kde je návrh změny specifikace veřejně dostupný (Brandner, [b.r.(d)]).

7.2 Samotná implementace

Pro přidání funkce break-out místností do aplikace Element Call budeme muset provést změny ve dvou repositářích – [matrix-js-sdk](#) (Matrix JavaScript SDK – více v části [6.2.1](#)) a [element-call](#) (samotná aplikace Element Call). Jak Element Call tak Matrix JavaScript SDK využívají programovací jazyk TypeScript (matrix.org, [b.r.(b)]; Element, [b.r.(c)]), což je nadstavba na JavaScriptu, která jej rozšiřuje o statické typování (Microsoft, [b.r.(b)]).

7.2.1 Matrix JavaScript SDK

Změny, které budeme v této části práce popisovat, lze nalézt [zde](#) (Brandner, [b.r.(a)]).

Prvním krokem bude přidání nové veřejné metody s názvem `createBreakoutRooms()` do třídy `MatrixClient`, kterou najdeme v souboru `/src/client.ts`. Tato metoda bude mít dva parametry – `parentRoomId`, což je ID místnosti, ze které break-out místnosti vytváříme, a `rooms`, což je seznam break-out místností, o kterých chceme informovat další klienty v místnosti:

```
export class MatrixClient extends TypedEventEmitter<
  EmittedEvents,
  ClientEventHandlerMap,
> {
  public async createBreakoutRooms(
    parentRoomId: string,
    rooms: BreakoutRoom[],
  ): Promise<ISendEventResponse>;
}
```


Metoda projde seznam `rooms` a vytvoří z něj `content` pro `m.breakout` state událost (více v části 7.1). Metoda `createBreakoutRooms()` state událost typu `m.breakout` následně odešle do místnosti s ID `parentRoomId` pomocí metody `this.sendStateEvent()` (Brandner, [b.r.(a)]).

Dále přidáme třídu `BreakoutRooms`, která umožňuje monitorovat break-out místnosti, na které odkazuje `m.breakout` state událost v dané Matrix místnosti (Brandner, [b.r.(a)]):

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> { /* ...implementace... */ }
```

V konstruktoru třídy si uložíme instanci `room`, což je místnost, ve které budeme monitorovat změny události typu `m.breakout`:

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  public constructor(private room: Room);
}
```

Zároveň voláním metody `room.addListener()` docílíme toho, že bude-li `room` emitovat událost typu `RoomEvent.Timeline`, bude volána metoda `this.onEvent()`:

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  public constructor(private room: Room) {
    room.addListener(RoomEvent.Timeline, this.onEvent);
  }
}
```

Posledním krokem v konstruktoru třídy je získání aktuální Matrix state události typu `m.breakout` pomocí metody `this.getBreakoutEvent()` a její zpracování pomocí metody `this.parseBreakoutEvent()` (Brandner, [b.r.(a)]).

Privátní metoda `getBreakoutEvent()` najde Matrix událost typu `m.breakout` v místnosti `this.room` a následně ji vrátí (Brandner, [b.r.(a)]):

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  private getBreakoutEvent(): MatrixEvent | null;
}
```

Privátní metoda `parseBreakoutEvent()` překonvertuje `content` state události typu `m.breakout` na seznam `BreakoutRoomWithSummary[]`, který následně vrátí:

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  private async parseBreakoutEvent(
    event: MatrixEvent,
  ): Promise<BreakoutRoomWithSummary[]>;
}
```

Tento seznam je lépe zpracovatelný při dalším využití, neboť je obohacen o tzv. *shrnutí místnosti* (*room summary*), jež obsahuje různá metadata jako název místnosti, která nejsou obsažena v samotné `m.breakout` state události; toto shrnutí místnosti získáme pomocí metody `getRoomSummary()` na třídě `MatrixClient` (Brandner, [\[b.r.\(a\)\]](#)).

Privátní metoda `onEvent` je tzv. *event handler*, který je volán, pokud je emitována událost typu `RoomEvent.Timeline`. Jejím jediným parametrem je `event` typu `MatrixEvent`. Tato metoda nejprve pomocí metody `event.getType()` zjistí, jestli je typ této Matrix události `m.breakout`, a pokud tomu tak je, využije k získání aktuálního seznamu breakout místností metod `this.getBreakoutEvent()` a `this.parseBreakoutEvent()`. Pokud došlo ke změně tohoto seznamu, je využita metoda `this.emit()` k emitování informací o změně (Brandner, [\[b.r.\(a\)\]](#)):

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  private onEvent: (event: MatrixEvent) => Promise<void>;
}
```

Veřejná metoda `getCurrentBreakoutRooms()` pak už jen vrací kopii privátního seznamu `this.currentBreakoutRooms`, která může být následně využita jinde (Brandner, [\[b.r.\(a\)\]](#)):

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  public getCurrentBreakoutRooms(): BreakoutRoomWithSummary[] | null;
}
```

V konstruktoru třídy `Room`, kterou najdeme v souboru `/src/models/room.ts`, vytvoříme instanci třídy `BreakoutRooms`, kterou uložíme do privátního člena třídy s názvem `breakoutRooms`, zároveň budeme reemitovat všechny události emitované `breakoutRooms`:

```
export class Room extends ReadReceipt<RoomEmittedEvents, RoomEventHandlerMap> {
  public constructor(
    public readonly roomId: string,
    public readonly client: MatrixClient,
    public readonly myUserId: string,
    private readonly opts: IOpts = {},
  ) {
    this.breakoutRooms = new BreakoutRooms(this);
    this.reEmitter.reEmit(this.breakoutRooms, [BreakoutRoomsEvent.RoomsChanged]);
  }
}
```

Přidáme také novou veřejnou metodu do třídy `Room` s názvem `getBreakoutRooms()`:

```
export class BreakoutRooms extends TypedEventEmitter<
  BreakoutRoomsEvent,
  BreakoutRoomsEventHandlerMap,
> {
  public getBreakoutRooms(): BreakoutRoomWithSummary[] | null;
}
```

Tato metoda zavolá `this.breakoutRooms.getCurrentBreakoutRooms()` a vrátí její návratovou hodnotu. Díky reemitování událostí pocházejících z `breakoutRooms` a metodě `getBreakoutRooms()` budou všichni s přístupem k instanci třídy `Room` mít přístupné i informace o existujících break-out místnostech (Brandner, [b.r.(a)]).

7.2.2 Element Call

Změny, které budeme v této části práce popisovat, lze nalézt zde (Brandner, [b.r.(b)]).

Většina změn v aplikaci Element Call spočívá převážně v úpravě a doplnění uživatelského rozhraní o prvky sloužící k vytváření a využívání break-out místností (Brandner, [b.r.(b)]). Element Call pro práci s uživatelským rozhraním využívá knihovnu React (Element, [b.r.(c)]). Ta nám umožňuje abstrahovat části práce s jazykem **HyperText Markup Language (HTML)** a JavaScriptem pomocí tzv. komponentů (a jiných struktur jako jsou např. *hooks*, či *contexts*). Zjednodušeně řečeno, komponent je JavaScript funkce, jejíž návratová hodnota bude později překonvertována do **HTML**, které bude zobrazeno na stránce. Komponenty mohou mít vnitřní logiku a parametry, které ovlivní jejich návratovou hodnotu (Meta Open Source, [b.r.]). (Samotné detaily změn uživatelského rozhraní zde nebudeme rozebírat, neboť nejsou příliš relevantní k tématu protokolu Matrix atp.)

Aplikace využívá koncové šifrování, které je založeno na jednom klíči, jenž je sdílený mezi uživateli pomocí URL (Brandner, [b.r.(c)]). Bude tedy nutné přidat mechanismus na sdílení těchto klíčů. Využijeme k tomu to-device messages (více v části 5.2.4). Komponent `BreakoutRoomModal` obsahuje funkci `onSubmit()`, která rozešle pomocí metody `client.sendToDevice()` klíče k nově vytvořeným break-out místnostem ostatním klientům¹⁰:

```
export const BreakoutRoomModal: FC<Props> = ({ client, roomId, open, onDismiss }) => {
  const onSubmit: () => Promise<void>;
};
```

Zároveň v kontextu `ClientContext`, jenž je v souboru `src/ClientContext.tsx`, přidáme nový event handler s názvem `onToDevice()`, který budeme volat, přijde-li nová to-device message:

```
export const ClientProvider: FC<Props> = ({ children }) => {
  const onToDevice: (event: MatrixEvent) => void;
};
```

Bude-li obsahovat informace o nových klíčích, uložíme je k dalšímu použití (Brandner, [b.r.(b)]).

Tímto je implementace break-out místností v aplikaci Element Call hotová.

¹⁰Metodu `sendToDevice()` využíváme pouze pro demonstrační účely – správně bychom měli využít metodu `encryptAndSendToDevices()`, aby data byla šifrovaná.

8 Budoucnost

Je těžké předpovídat, jak se budou **VoIP** aplikace vyvíjet v příštích několika letech, ale některá fakta nám mohou mírně napovědět – na ně se v této části podíváme.

8.1 Využití MatrixRTC v non-VoIP aplikacích

Přestože historicky hlavním účelem MatrixRTC je **VoIP** pomocí protokolu Matrix, **WebRTC** umožňují přenos arbitrárních informací, je tedy možné MatrixRTC využít i pro jiné účely.

8.1.1 Third Room – virtuální světy postavené na Matrixu

Jedním z projektů, který využívá MatrixRTC, je **Third Room** (The Matrix.org Foundation, [b.r.(g)]). Jedná se o vizi otevřeného Metaverse¹¹, které používá Matrix jako svou signalling vrstvu pro virtuální 3D světy.

Third Room využívá engine jménem *Manifold*, který je založený na technologiích jako je **Three.js**, **bitECS** a **Rapier**. Jako své Matrix SDK využívá **Hydrogen SDK** (The Matrix.org Foundation, [b.r.(g)]).

8.1.2 TheBoard – virtuální tabule

TheBoard je nástroj pro spolupráci, který umožňuje svým uživatelům kreslit na virtuální tabuli v reálném čase (Timo, [b.r.]). TheBoard aktuálně MatrixRTC nevyužívá, ale je to vhodný kandidát pro jeho využití, neboť by projekt mohl využít **WebRTC** datové kanály k téměř okamžité komunikaci v reálném čase.

8.2 Web Transport – alternativa k WebRTC

Web Transport je experimentální protokol pro klient-server komunikaci založenou na **UDP**. Představuje alternativu k **WebRTC**, neboť rovněž umožňuje posílání médií a arbitrárních dat s nízkým zpožděním (World Wide Web Consortium, [b.r.]).

8.3 Digital Markets Act

Digital Markets Act (DMA) je legislativa, jejímž cílem je vytvořit více kompetitivní prostředí mezi tzv. *gatekeepers*, což jsou velcí poskytovatelé klíčových služeb jako je chat,

¹¹ *Metaverse* je virtuální svět, ve kterém mohou lidé trávit svůj čas prací, zábavou, socializováním atp. Termín *Metaverse* vytvořil Neal Stephenson ve své sci-fi knize *Snow Crash* (Tucci, 2022)

VoIP atp.; příkladem je Meta, Apple, Google a Microsoft. DMA nařizuje, aby tito poskytovatelé umožnili přístup k jejich API, což by umožnilo lepší interoperabilitu mezi službami. DMA zároveň nařizuje, že u služeb, které poskytují E2EE, by ho API měla zachovávat (Pape, 2022).

Aktuálně je vývoj Matrix přemostění komplikován tím, že gatekeepers nemusí mít veřejná a zdokumentovaná API, DMA by ale mohlo tuto situaci změnit a umožnit snadnější vývoj přemostění, což by mohlo napomoci protokolu Matrix, aby byl využíván větším množstvím běžných uživatelů.

9 Závěr

Cílem této práce bylo rozebrat poměrně nezmapovanou oblast – fungování **VoIP** aplikací a fungování protokolu Matrix. Dalším cílem bylo vytvořit nový návrh doplnění specifikace protokolu Matrix a implementovat break-out rooms do Matrix JavaScript SDK a aplikace Element Call.

Na prvních několika stránkách textu jsme rozebrali fungování protokolu **WebRTC**, který je základem mnoha **VoIP** aplikací. Ukázali jsme si, z jakých dílčích protokolů se tento protokol skládá, a vysvětlili jsme si, jak fungují.

V další části práce jsme si ukázali, jak se využívá protokol **WebRTC** pro skupinové volání – rozebrali jsme různé modely propojení více uživatelů a jejich výhody a nevýhody.

V následující části jsme si stručně představili některé jiné existující **VoIP** technologie jako Jitsi a Zoom.

Poté jsme rozebrali, jak funguje protokol Matrix. Vysvětlili jsme, jak funguje jeho otevřená specifikace a jakým způsobem se vyvíjí. Následně jsme si prošli jednotlivá primitiva, ze kterých je protokol Matrix složený. Podívali jsme se na několik implementací klientů a homeserverů. Také jsme vysvětlili, jak fungují přemostění a end-to-end šifrování.

Následně jsme probrali protokol MatrixRTC (část protokolu Matrix zaměřená na **VoIP**), jeho využití Matrix primitiv a samotné fungování hovorů mezi dvěma a více uživateli. Prošli jsme též implementace klientských **SDK**. V neposlední řadě jsme si ukázali koncept foci a jejich fungování.

V praktické části práce jsme navrhli doplnění protokolu Matrix o funkci break-out rooms, která zatím není jeho součástí, a následně tento návrh implementovali do Matrix JavaScript SDK a aplikace Element Call.

V poslední části práce jsme zaměřili na potenciální využití MatrixRTC mimo **VoIP** aplikace a obecný vývoj v oblasti protokolu Matrix a **VoIP** aplikací.

Implementace break-out rooms je plně funkční, ale před dalšími kroky musí být samozřejmě řádně otestována. Následně bude možné, aby změny Matrix JavaScript SDK a Element Call byly přijaty. Na poli protokolu Matrix by dalším krokem bylo posunutí návrhu změny specifikace do fáze **FCP**.

Základním přínosem mé práce bylo úspěšné doplnění specifikace MatrixRTC o dosud chybějící část – break-out rooms. Tím byly rozšířeny nejen funkce protokolu, ale i přibýlo možností, jak jej využít – break-out rooms přispívají k tomu, aby protokol (i aplikace Element Call) byl atraktivnější např. pro sektor školství.

Dalšími důležitými přínosy této práce jsou zmapování WebRTC a Matrix v českém jazyce, což doposud nebylo učiněno, a přehled fungování protokolu MatrixRTC (takový přehled, pokud je mi známo, nebyl zpracován vůbec).

10 Seznam zkratek

- Advanced Encryption Standard (AES)** Aktuálně nejvyžívanější symetrická bloková šifra (Paar; Pelzl, 2009). 24
- Advanced Research Projects Agency Network (ARPANET)** Experimentální počítačová síť, která byla předchůdcem internetu (Featherly, 2023). 1
- Application Programming Interface (API)** Rozhraní, které umožňuje dvěma komponentům spolu komunikovat pomocí definovaných protokolů (Amazon Web Services, [b.r.]). 3, 18, 26, 41
- Datagram Transport Layer Security (DTLS)** Protokol, který generuje klíče pro šifrování dat posílaných přes protokol UDP. (Viz 2.4.1.) 9, 19, 44
- Decentralized Messaging Layer Security (DMLS)** Decentralizovaná verze šifrovacího protokolu MLS. (Viz 5.6.) 24
- Digital Markets Act (DMA)** Legislativa, která má za cíl zamezit tvoření monopolů. (Viz 8.3.) 40, 41
- Dual-Tone Multi-Frequency (DTMF)** Systém, ve kterém má každá číslice přiřazenou dvojici tónů. Tyto tóny se pak používají pro komunikaci mezi zařízeními (Dstny, 2019). 25
- End-to-end encryption** Metoda šifrování, kde třetí strany nemohou číst data, neboť klíče mají jen koncové strany. Příkladem je mnoho aplikací pro chat, ve kterých server nemůže číst zprávy uživatelů, neboť klíče mají jen klienti daných uživatelů (IBM, [b.r.]). 3, 9, 16, 23, 41
- Final Comment Period (FCP)** Doba před přijetím MSC do specifikace protokolu Matrix, kdy je prostor na poslední komentáře. (Viz 5.1.) 21, 42
- HyperText Markup Language (HTML)** Jedná se o jazyk popisující strukturu a obsah webu (MDN contributors, 2023a). 39
- Interactive Connectivity Establishment (ICE)** Protokol, který hledá nejlepší způsob, jak propojit dva počítače. (Viz 2.1.4.) 3, 7–10, 12, 13, 18, 25, 26, 28, 31
- Matrix Spec Change (MSC)** Formální návrh pro změnu specifikace protokolu Matrix. (Viz 5.1.) 21, 24, 25, 43

Messaging Layer Security (MLS) Protokol pro šifrování zpráv ve skupinách dvou a více uživatelů (Messaging Layer Security, [b.r.]). (Viz 5.6.) 24, 43

Multipoint Control Unit (MCU) Server, který přijímá streamy jednotlivých uživatelů, spojuje je v jeden a posílá jej uživatelům zpět. (Viz 3.2.1.) 15, 17, 31

Network Address Traversal (NAT) Metoda pro mapování privátních adres v podsíti routeru na jednu veřejnou adresu routeru. (Viz 2.1.1.) 4–8, 45

Peer-to-peer Peer-to-peer spojení dvou počítačů je takové spojení, kde k výměně dat není třeba žádný server (Merriam-Webster, 2023). 3, 14

Public Switched Telephone Network (PSTN) Běžná veřejná telefonní síť (Johnson, 2018). 28

Real Time Messaging Protocol (RTMP) Protokol pro přeposílání audia a videa přes internet (Restream Team, 2023). 18

Real-time Transport Protocol (RTP) Protokol určený pro přenos audio a videa přes internet. (Viz 2.2.1.) 8, 9, 15, 44

RTP Control Protocol (RTCP) Protokol pro přeposílání metadat o spojení pomocí protokolu RTP. (Viz 2.2.2.) 8, 11

Secure Real-time Transport Protocol (SRTP) Protokol využívající externí šifrovací klíče vygenerované DTLS k šifrování RTP paketů. (Viz 2.4.2.) 9, 11, 19

Selective Forwarding Unit (SFU) Server, který přijímá streamy od uživatelů a jejich podmnožinu posílá zpět. (Viz 3.2.2.) 16–18, 31

Session Description Protocol (SDP) Protokol, který specifikuje, jak si mají dva klienti vyměňovat informace o jejich veřejných adresách, podporovaných kodecích atp. (Viz 2.5.) 9–12, 25–27

Session Initiation Protocol (SIP) Protokol, který specifikuje komunikaci se signalling serverem. (Viz 4.1.) 18

Session Traversal Utilities for NAT (STUN) Protokol, který umožňuje získání veřejné IP adresy routeru, za kterým se klient nachází. (Viz 2.1.2.) 5–8

Software Development Kit (SDK) Skupina nástrojů určená pro zjednodušení vývoje na dané platformě (Red Hat, 2020). 18, 22, 42

Spec Core Team (SCT) Tým, který se stará o specifikaci protokolu Matrix a schvaluje její změny. (Viz 5.1.) 21

Stream Control Transmission Protocol (SCTP) Protokol určený pro přeposílání libovolných dat přes WebRTC. (Viz 2.3.) 8, 11, 13

Transmission Control Protocol (TCP) Protokol pro přenos dat mezi počítači se zárukou doručení (The Editors of Encyclopædia Britannica, 2023a). 5, 6, 9, 18, 45

Transport Layer Security (TLS) Protokol určený pro šifrování dat posílaných protokolem TCP. (Microsoft, 2023). 9

Traversal Using Relays around NAT (TURN) Metoda, která využívá proxy (prostředníky) pro přeposílání dat. Využívá se, pokud nejde jinak obejít problémy vytvořené NAT. (Viz 2.1.3.) 6, 7

User Datagram Protocol (UDP) Protokol pro přenos dat mezi počítači bez záruky doručení (GeeksforGeeks, 2023). 6, 9, 11, 40, 43

Voice over Internet Protocol (VoIP) Komunikační technologie sloužící pro telefonní spojení pomocí internetu (The Editors of Encyclopædia Britannica, 2023b). 1, 2, 18, 19, 25, 40–42, 45

Web Real-Time Communication (WebRTC) Protokol pro provádění VoIP hovorů na webu. (Viz 2.) 1–3, 8–12, 14, 18, 26, 31, 40, 42, 45

11 Seznam obrázků

1	NAT (PU – veřejná IP adresa, PR – privátní IP adresa)	4
2	STUN (PU – veřejná IP adresa, PR – privátní IP adresa)	5
3	TURN (PU – veřejná IP adresa, PR – privátní IP adresa)	6
4	Signalling	9
5	Full-mesh	14
6	Multipoint Control Unit	15
7	Selective Control Unit	16
8	Simulcast	17
9	Matrix federace	20
10	Federace Matrix foci	31
11	Federace Matrix foci: Charlie se připojuje do hovoru	32
12	Federace Matrix foci: Dave se připojuje do hovoru	33

12 Reference

- 8X8. *Introduction*. Online. Dostupné z: <https://jitsi.github.io/handbook/docs/intro/>. [citováno 2023-04-19].
- 8X8. *Jitsi Free & Open Source Video Conferencing Projects*. Online. Dostupné z: <https://jitsi.org/projects/>. [citováno 2023-03-12].
- ABRELL, Moritz (2020). *New Ways of Communicating – When End-to-End-Encryption Gains a New Meaning*. IT Security Know-How.
- AGUSTINA; EMA; SUHARYA; TOTO (2021). *Zoom breakout rooms for students' collaborative skill enhancement in history learning during Covid-19 outbreak*. International Journal of Research in Counseling and Education, roč. 5, s. 41–46. Dostupné z DOI: [10.24036/00430za0002](https://doi.org/10.24036/00430za0002).
- AHMED; KHALID (2021). *The Impact of Utilizing Breakout Rooms in ESL Distance Learning from Students Perspectives*.
- AMAZON WEB SERVICES. *What is an API (Application Programming Interface)?*. Online. Dostupné z: <https://aws.amazon.com/what-is/api>. [citováno 2023-11-11].
- BAKER, David; HODGSON, Matthew; DEEPBLUEV7.X; BRANDNER, Šimon; CHATHI, Hubert; HOFF, Richard van der. *MSC2746: Improved VoIP Signalling*. Online. Dostupné z: <https://github.com/matrix-org/matrix-spec-proposals/pull/2746>. [citováno 2023-04-20].
- BEGEN, Ali C.; KYZIVAT, Paul; PERKINS, Colin; HANDLEY, Mark J. (2021). *SDP: Session Description Protocol*. Online. 2021-01. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc8866>. [citováno 2023-04-22].
- BOUND, Leeward; DUBOIS, Sean; VOLUNTAS. *WebRTC for the Curious: Go beyond the APIs*. Online. Dostupné z: <https://webrtcforthe curious.com/>. [citováno 2023-04-18].
- BRANDNER, Šimon. *Add support for break-out rooms*. Online. Dostupné z: <https://github.com/matrix-org/matrix-js-sdk/pull/3753>. [citováno 2023-10-21].
- BRANDNER, Šimon. *Add support for break-out rooms*. Online. Dostupné z: <https://github.com/vector-im/element-call/pull/1615>. [citováno 2023-10-21].
- BRANDNER, Šimon. *Complete SPA E2EE work*. Online. Dostupné z: <https://github.com/vector-im/element-call/pull/1344>. [citováno 2023-10-21].

- BRANDNER, Šimon. *MSC3985: Break-out rooms*. Online. Dostupné z: <https://github.com/matrix-org/matrix-spec-proposals/pull/3985>. [citováno 2023-10-19].
- BRANDNER, Šimon; ASOKAN, Tulir. *MSC3077: Support for multi-stream VoIP*. Online. Dostupné z: <https://github.com/matrix-org/matrix-spec-proposals/pull/3077>. [citováno 2023-04-20].
- BRANDNER, Šimon; MORGAN, Andrew. *MSC3291: Muting in VoIP calls*. Online. Dostupné z: <https://github.com/matrix-org/matrix-spec-proposals/pull/3291>. [citováno 2023-04-20].
- BRUAROEY, Jan-Ivar (2019). *Perfect negotiation in WebRTC*. Online. 2019-04-07. Dostupné z: <https://blog.mozilla.org/webrtc/perfect-negotiation-in-webrtc/>. [citováno 2023-04-21].
- BURA, Ajay. *Cinny*. Online. Dostupné z: <https://cinny.in/>. [citováno 2023-03-12].
- BUSCH, Zack (2021). *What Is Port Forwarding? - A Beginner's Guide*. Online. 2021-01-01. Dostupné z: <https://learn.g2.com/port-forwarding>. [citováno 2023-09-20].
- CERN. *The birth of the Web*. Online. Dostupné z: <https://www.home.cern/science/computing/birth-web>. [citováno 2023-03-12].
- CINNY. *Cinny*. Online. Dostupné z: <https://github.com/cinnyapp/cinny>. [citováno 2023-03-12].
- DEBARI, Joe (2021). *VoIP Stats, VoIP Trends, and the Impact of COVID-19*. Online. 2021-10-06. Dostupné z: <https://www.onsip.com/voip-resources/industry-news-trends/voip-stats-voip-trends-and-the-impact-of-covid-19>. [citováno 2023-03-12].
- DIGIFONE. *What You Might Not Know About the History of VoIP*. Online. Dostupné z: <https://digifone.net/what-you-might-not-know-about-the-history-of-voip/>. [citováno 2023-03-12].
- DISCORD. *What Is Discord?*. Online. Dostupné z: <https://discord.com/safety/360044149331-what-is-discord>. [citováno 2023-04-19].
- DSTNY (2019). *DTMF*. Online. 2019-12-10. Dostupné z: <https://www.dstny.se/en/glossary/dtmf/>. [citováno 2023-04-22].

ELEMENT. *Element*. Online.

Dostupné z: <https://element.io/>. [citováno 2023-03-12].

ELEMENT. *Element*. Online.

Dostupné z: <https://github.com/vector-im/element-web>. [citováno 2023-04-19].

ELEMENT. *Element Call*. Online.

Dostupné z: <https://github.com/vector-im/element-call>. [citováno 2023-04-19].

ELEMENT. *Element Desktop*. Online. Dostupné z:

<https://github.com/vector-im/element-desktop/>. [citováno 2023-03-12].

ELEMENT. *Element iOS*. Online.

Dostupné z: <https://github.com/vector-im/element-ios/>. [citováno 2023-03-12].

ELEMENT. *Element X Android*. Online. Dostupné z:

<https://github.com/vector-im/element-x-android/>. [citováno 2023-03-12].

ELEMENT. *Element X iOS*. Online. Dostupné z:

<https://github.com/vector-im/element-x-ios/>. [citováno 2023-03-12].

ELEMENT. *Hydrogen*. Online.

Dostupné z: <https://github.com/vector-im/hydrogen-web>. [citováno 2023-03-12].

Element Android. Online. Dostupné z:

<https://github.com/vector-im/element-android/>. [citováno 2023-03-12].

FAMEDLY. *FluffyChat*. Online.

Dostupné z: <https://gitlab.com/famedly/fluffychat>. [citováno 2023-03-12].

FEAR, Krille. *FluffyChat*. Online.

Dostupné z: <https://fluffychat.im/>. [citováno 2023-03-12].

FEATHERLY, Kevin (2023). *ARPANET*. Online.

Dostupné z: <https://www.britannica.com/topic/ARPANET>. [citováno 2023-11-11].

FRY, Jeff (2021). *SIP Protocol Overview, Performance, Monitoring*. Online. 2021-08-16.

Dostupné z: <https://www.extrahop.com/company/blog/2016/sip-protocol/>.
[citováno 2023-04-19].

GEEKSFORGEES (2023). *User Datagram Protocol (UDP)*. Online. 2023-05-11.

Dostupné z: <https://www.geeksforgeeks.org/user-datagram-protocol-udp/>.
[citováno 2023-11-11].

GILLIS, Alexander S. (2022). *codec*. Online. Dostupné z: <https://www.techtarget.com/searchunifiedcommunications/definition/codec>.

[citováno 2023-09-14].

GITHUB. *About pull requests*. Online.

Dostupné z: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>. [citováno 2023-03-09].

GITHUB. *GitHub-flavoured Markdown*. Online.

Dostupné z: <https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>. [citováno 2023-03-09].

GOOGLE (2020). *Data channels*. Online. 2020-08-03. Dostupné z:

<https://webrtc.org/getting-started/data-channels>. [citováno 2023-04-21].

GOOGLE. *Peer connections*. Online. Dostupné z:

<https://webrtc.org/getting-started/peer-connections>. [citováno 2023-03-17].

GOOGLE. *Real-time communication for the web*. Online.

Dostupné z: <https://webrtc.org/>. [citováno 2023-03-12].

GROZEV, Boris.

Efficient and scalable video conferences with selective forwarding units and webRTC.
Dis. pr. Université de Strasbourg.

GRUBER, John. *Markdown*. Online. Dostupné z:

<https://daringfireball.net/projects/markdown/>. [citováno 2023-03-09].

HADDOCK, Stephen (2010).

Resilient Network Interconnect: Split Brain Avoidance. Online. 2010-12-09.

Dostupné z: <https://www.ieee802.org/1/files/public/docs2010/new-haddock-RNNI-split-brain-avoidance-1210-v1.pdf>. [citováno 2023-03-18].

HALAVAI, Alexander (2022). *codec*. Online.

Dostupné z: <https://www.britannica.com/technology/codec>.
[citováno 2023-09-14].

HANCKE, Philipp (2019).

How Zoom's web client avoids using WebRTC (DataChannel Update). Online.

2019-09-08. Dostupné z: <https://webrtcchacks.com/zoom-avoids-using-webrtc/>.
[citováno 2023-04-19].

HANCKE, Philipp (2020).

True End-to-End Encryption with WebRTC Insertable Streams. Online. 2020-04-12.

Dostupné z: <https://webrtcchacks.com/true-end-to-end-encryption-with-webrtc-insertable-streams/>. [citováno 2023-03-17].

- HODGSON, Matthew; BRANDNER, Šimon; ABRAMOV, Daniel; BAKER, David.
MSC3401: Native Group VoIP Signalling. Online. Dostupné z:
<https://github.com/matrix-org/matrix-spec-proposals/pull/3898>. [citováno 2023-03-12].
- HODGSON, Matthew; BRANDNER, Šimon; BAKER, David; SCHWENDIG, Enrico.
Implement MSC3898: Native Matrix VoIP signalling for cascaded SFUs. Online.
Dostupné z: <https://github.com/matrix-org/matrix-js-sdk/pull/2423/>.
[citováno 2023-04-19].
- HODGSON, Matthew; LONG, Robert; TOWNSEND, Robin; BRANDNER, Šimon.
MSC3401: Native Group VoIP Signalling. Online. Dostupné z:
<https://github.com/matrix-org/matrix-spec-proposals/pull/3401/>. [citováno 2023-03-12].
- HOLMBERG, Christer; SHPOUNT, Roman; LORETO, Salvatore;
CAMARILLO, Gonzalo (2021). *Session Description Protocol (SDP) Offer/Answer Procedures for Stream Control Transmission Protocol (SCTP) over Datagram Transport Layer Security (DTLS) Transport*. Online. 2021-01. Dostupné z:
<https://datatracker.ietf.org/doc/html/rfc8841>. [citováno 2023-04-27].
- IBM. *What is end-to-end encryption?*. Online. Dostupné z:
<https://www.ibm.com/topics/end-to-end-encryption>. [citováno 2023-09-14].
- INFRARED5 (2021).
3 Key Approaches for Scaling WebRTC: SFU, MCU, and XDN. Online. 2021-01-20.
Dostupné z: <https://www.red5pro.com/blog/3-key-approaches-for-scaling-webrtc-sfu-mcu-and-xdn/>. [citováno 2023-03-17].
- JENNINGS, Cullen Fluffy; AUDET, Francois (2007). *Network Address Translation (NAT) Behavioral Requirements for Unicast UDP*. Online. 2007-01. Dostupné z:
<https://datatracker.ietf.org/doc/html/rfc4787>. [citováno 2023-09-14].
- JOHNSON, Cameron (2018). *PSTN*. Online. 2018-11-19. Dostupné z:
<https://www.nextiva.com/blog/what-is-pstn.html>. [citováno 2023-04-22].
- KESBY, Rebecca. *How the world's first webcam made a coffee pot famous*. Online.
Dostupné z: <https://www.bbc.com/news/technology-20439301>. [citováno 2023-03-12].
- KOESTERS, Timo. *Conduit*. Online.
Dostupné z: <https://github.com/timokoesters/conduit/>. [citováno 2023-03-13].

- LEMMER-WEBBER, Christine; TALLON, Jessica; SHEPHERD, Erin; GUY, Amy; PRODROMOU, Evan (2018). *ActivityPub*. Online. 2018-01-23.
Dostupné z: <https://www.w3.org/TR/activitypub/>. [citováno 2023-03-12].
- LEVENT-LEVI, Tsahi (2021).
What is WebRTC P2P mesh and why it can't scale?. Online. 2021-01-20.
Dostupné z: <https://bloggeek.me/webrtc-p2p-mesh>. [citováno 2023-03-17].
- LEVENT-LEVI, Tsahi; LIGHT, John.
Will the IPv6 result in the death of STUN and ICE?. Online.
Dostupné z: <https://www.quora.com/Will-the-IPv6-result-in-the-death-of-STUN-and-ICE>. [citováno 2023-04-20].
- LIVEKIT. *LiveKit*. Online. Dostupné z: <https://livekit.io/>. [citováno 2023-04-19].
- LIVEKIT. *LiveKit: Real-time video, audio and data for developers*. Online.
Dostupné z: <https://github.com/livekit/livekit>. [citováno 2023-04-19].
- LOVEJOY, Ben (2022). *Messaging interoperability would be a nightmare for tech giants, but a dream for consumers*. Online. 2022-03-25.
Dostupné z: <https://9to5mac.com/2022/03/25/messaging-interoperability-nightmare-dream/>. [citováno 2023-03-12].
- MATRIX CONSTRUCT. *Construct*. Online. Dostupné z:
<https://github.com/matrix-construct/construct/>. [citováno 2023-03-13].
- MATRIX-ORG. *Olm*. Online.
Dostupné z: <https://gitlab.matrix.org/matrix-org/olm>. [citováno 2023-03-13].
- MATRIX.ORG. *Dendrite*. Online.
Dostupné z: <https://github.com/vector-im/dendrite/>. [citováno 2023-03-13].
- MATRIX.ORG. *Matrix JavaScript SDK*. Online. Dostupné z:
<https://github.com/matrix-org/matrix-js-sdk>. [citováno 2023-03-12].
- MATRIX.ORG. *matrix-react-sdk*. Online. Dostupné z:
<https://github.com/matrix-org/matrix-react-sdk>. [citováno 2023-04-19].
- MATRIX.ORG. *matrix-rust-sdk*. Online. Dostupné z:
<https://github.com/matrix-org/matrix-rust-sdk/>. [citováno 2023-03-13].
- MATRIX.ORG. *Synapse*. Online.
Dostupné z: <https://github.com/matrix-org/synapse/>. [citováno 2023-03-13].
- MATRIX.ORG. *Waterfall*. Online.
Dostupné z: <https://github.com/matrix-org/waterfall/>. [citováno 2023-04-19].

- MCDOWALL, Mike (2015). *How a simple 'hello' became the first message sent via the Internet*. Online. 2015-02-09. Dostupné z: <https://www.pbs.org/newshour/science/internet-got-started-simple-hello>. [citováno 2023-03-12].
- MDN CONTRIBUTORS (2023). *HTML: HyperText Markup Language*. Online. 2023-07-18. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/HTML>. [citováno 2023-11-11].
- MDN CONTRIBUTORS (2023). *MediaStream*. Online. 2023-02-20. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/MediaStream>. [citováno 2023-04-21].
- MDN CONTRIBUTORS (2023). *MediaStreamTrack*. Online. 2023-02-20. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack>. [citováno 2023-04-21].
- MDN CONTRIBUTORS. *RTCIceCandidate: toJSON() method*. Online. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/RTCIceCandidate/toJSON>. [citováno 2023-04-20].
- MDN CONTRIBUTORS. *Signaling and video calling*. Online. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Signaling_and_video_calling. [citováno 2023-09-14].
- MERRIAM-WEBSTER (2023). *peer-to-peer*. Online. 2023-09-11. Dostupné z: <https://www.merriam-webster.com/dictionary/peer-to-peer>. [citováno 2023-09-14].
- MESSAGING LAYER SECURITY. *What is Messaging Layer Security (MLS)?*. Online. Dostupné z: <https://messaginglayersecurity.rocks/>. [citováno 2023-11-11].
- META OPEN SOURCE. *React*. Online. Dostupné z: <https://react.dev/>. [citováno 2023-10-21].
- MICROSOFT. *Limits and specifications for Microsoft Teams*. Online. Dostupné z: <https://learn.microsoft.com/en-us/microsoftteams/limits-specifications-teams>. [citováno 2023-03-12].
- MICROSOFT (2023). *Transport Layer Security protocol*. Online. 2023-02-08. Dostupné z: <https://learn.microsoft.com/en-us/windows-server/security/tls/transport-layer-security-protocol>. [citováno 2023-11-11].

- MICROSOFT. *TypeScript is JavaScript with syntax for types*. Online.
Dostupné z: <https://www.typescriptlang.org/>. [citováno 2023-10-20].
- MICROSOFT. *Use breakout rooms in Microsoft Teams meetings*. Online. Dostupné z: <https://support.microsoft.com/en-gb/office/use-breakout-rooms-in-microsoft-teams-meetings-7de1f48a-da07-466c-a5ab-4ebace28e461>. [citováno 2023-10-19].
- MOOK, Nate (2005). *Microsoft, Yahoo to Link IM Networks*. Online. 2005-10-12.
Dostupné z: <https://betanews.com/2005/10/12/microsoft-yahoo-to-link-im-networks/>. [citováno 2023-03-12].
- MYŠKA, Matěj; POLČÁK, Radim; ŠAVELKA, Jaromír; KYNCL, Libor;
SVIRÁKOVÁ, Iveta. *Veřejné licence v České republice* online. [citováno 2023-06-20].
ISBN 978-80-210-7193-3.
Dostupné z: <https://www.fi.muni.cz/studies/verejne-licence.pdf>.
- NHEKO-REBORN. *nheko*. Online.
Dostupné z: <https://github.com/Nheko-Reborn/nheko>. [citováno 2023-03-12].
- OPENJS FOUNDATION AND ELECTRON CONTRIBUTORS (2020).
What is an SDK?. Online. 2020-06-10.
Dostupné z: <https://www.electronjs.org>. [citováno 2023-03-12].
- PAAR, Christof; PELZL, Jan.
Understanding Cryptography: A Textbook for Students and Practitioners.
ISBN 978-3-642-04101-3.
- PAPE, Amandine Le (2022). *A guide to navigating the Digital Markets Act*. Online.
2022-04-08. Dostupné z: <https://element.io/blog/a-guide-to-navigating-the-digital-markets-act/>. [citováno 2023-04-19].
- PETIT-HUGUENIN, Marc; NANDAKUMAR, Suhas; HOLMBERG, Christer;
KERÄNEN, Ari; SHPOUNT, Roman (2021). *Session Description Protocol (SDP) Offer/Answer Procedures for Interactive Connectivity Establishment (ICE)*. Online.
2021-01. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc8839>.
[citováno 2023-09-20].
- PION. *Pion WebRTC*. Online.
Dostupné z: <https://github.com/pion/webrtc>. [citováno 2023-04-20].
- RALSTON, Travis. *MSC3414: Encrypted state events*. Online. Dostupné z: <https://github.com/matrix-org/matrix-spec-proposals/pull/3414>. [citováno 2023-03-13].

- RED HAT (2020). *What is an SDK?*. Online. 2020-06-10. Dostupné z: <https://www.redhat.com/en/topics/cloud-native-apps/what-is-SDK>. [citováno 2023-03-12].
- RESTREAM TEAM (2023). *RTMP streaming: Everything you need to know*. Online. 2023-05-05. Dostupné z: <https://restream.io/blog/rtmp-streaming/>. [citováno 2023-11-11].
- SHONTELL, Alyson (2013). *What You Might Not Know About the History of VoIP*. Online. 2013-04-23. Dostupné z: <https://finance.yahoo.com/news/the-first-ever-email--the-first-tweet--and-12-other-famous-internet-firsts-181209886.html>. [citováno 2023-03-12].
- SCHULZRINNE, Henning; CAMARILLO, Gonzalo (2010). *The Session Description Protocol (SDP) Grouping Framework*. Online. 2010-06. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc5888>. [citováno 2023-04-22].
- TÄNAVSUU, Toivo (2018). *“How can they be so good?”: The strange story of Skype*. Online. 2018-09-03. Dostupné z: <https://arstechnica.com/information-technology/2018/09/skypes-secrets/2/>. [citováno 2023-03-12].
- THE EDITORS OF ENCYCLOPÆDIA BRITANNICA (2023). *TCP/IP*. Online. Dostupné z: <https://www.britannica.com/technology/TCP-IP>. [citováno 2023-11-11].
- THE EDITORS OF ENCYCLOPÆDIA BRITANNICA (2023). *VoIP*. Online. Dostupné z: <https://www.britannica.com/technology/VoIP>. [citováno 2023-11-11].
- THE MATRIX.ORG FOUNDATION. *Are we MLS yet?*. Online. Dostupné z: <https://arewemlsyet.com/>. [citováno 2023-03-13].
- THE MATRIX.ORG FOUNDATION. *Frequently Asked Questions*. Online. Dostupné z: <https://matrix.org/faq>. [citováno 2023-03-07].
- THE MATRIX.ORG FOUNDATION. *Matrix*. Online. Dostupné z: <https://matrix.org/>. [citováno 2023-03-07].
- THE MATRIX.ORG FOUNDATION. *Matrix Bridges*. Online. Dostupné z: <https://matrix.org/bridges>. [citováno 2023-03-13].
- THE MATRIX.ORG FOUNDATION. *Matrix Specification*. Online. Dostupné z: <https://spec.matrix.org/latest>. [citováno 2023-03-12].

- THE MATRIX.ORG FOUNDATION. *Spec Change Proposals*. Online.
Dostupné z: <https://spec.matrix.org/proposals/>. [citováno 2023-03-09].
- THE MATRIX.ORG FOUNDATION. *Third Room*. Online.
Dostupné z: <https://thirdroom.io/>. [citováno 2023-04-19].
- TIMO. *TheBoard*. Online.
Dostupné z: <https://github.com/toger5/TheBoard>. [citováno 2023-04-19].
- TUCCI, Linda (2022).
What is the metaverse? An explanation and in-depth guide. Online. 2022-11-18.
Dostupné z: <https://www.techtarget.com/whatis/feature/The-metaverse-explained-Everything-you-need-to-know>. [citováno 2023-04-22].
- ULLAL, Aaron (2021). *What powers Google Meet and Microsoft Teams? WebRTC Demystified / Step By Step Tutorial*. Online. 2021-03-23. Dostupné z: <https://levelup.gitconnected.com/what-powers-google-meet-and-microsoft-teams-webrtc-demystified-step-by-step-tutorial-e0cb422010f7>. [citováno 2023-03-12].
- VASS, Jozsef (2018). *How discord handles two and half million concurrent voice users using webrtc*. Online. 2018-09-10.
Dostupné z: <https://discord.com/blog/how-discord-handles-two-and-half-million-concurrent-voice-users-using-webrtc>. [citováno 2023-03-12].
- WALLIS, Jerry (2023). *How Does Zoom Work? Video Conferencing Technology Explored — The Tech Behind Series*. Online. 2023-02-15. Dostupné z: <https://webo.digital/blog/how-does-zoom-work-video-conferencing-tech/>. [citováno 2023-04-19].
- WEBRTC.RS. *WebRTC.rs*. Online.
Dostupné z: <https://github.com/webrtc-rs/webrtc>. [citováno 2023-04-20].
- WESTERLUND, Magnus (2011). *Glare Handling in WebRTC Signalling*. Online. 2011-11-12.
Dostupné z: <https://www.ietf.org/proceedings/82/slides/rwcweb-10.pdf>. [citováno 2023-03-18].
- WORLD WIDE WEB CONSORTIUM. *WebTransport Explainer*. Online.
Dostupné z: <https://github.com/w3c/webtransport/blob/main/explainer.md>. [citováno 2023-04-19].
- XMPP STANDARDS FOUNDATION. *The universal messaging standard*. Online.
Dostupné z: <https://xmpp.org/>. [citováno 2023-03-12].

XMPP STANDARDS FOUNDATION. *WebRTC*. Online.

Dostupné z: <https://xmpp.org/uses/webrtc/>. [citováno 2023-04-19].

YU, Laura (2018). *NAT: Why Do We Need It?*. Online. 2018-08-23. Dostupné z:

https://medium.com/@laurayu_653/nat-why-do-we-need-it-f0230bb7d06f.
[citováno 2023-10-02].

ZEAL, Marcus. *Why Do Web Developers Hate Internet Explorer*. Online.

Dostupné z: <https://www.zealoussites.com/blog/why-do-web-developers-hate-internet-explorer>. [citováno 2023-03-12].

ZHAO, David (2021). *An Introduction to WebRTC Simulcast*. Online. 2021-08-26.

Dostupné z: <https://blog.livekit.io/an-introduction-to-webrtc-simulcast-6c5f1f6402eb/>. [citováno 2023-03-17].

ZOOM VIDEO COMMUNICATIONS (2023). *Enabling meeting breakout rooms*. Online. 2023-08-02.

Dostupné z: <https://support.zoom.us/hc/en-us/articles/206476093-Enabling-meeting-breakout-rooms>. [citováno 2023-10-19].

ZOOM VIDEO COMMUNICATIONS. *Zoom*. Online.

Dostupné z: <https://zoom.us/>. [citováno 2023-04-19].